

Interpolable Nyquist criterion: fast stability charts and rightmost-root estimation for linear time-delay systems via adaptive phase integration

Daniel Bachrathy^a

^a*Department of Applied Mechanics, Faculty of Mechanical Engineering, Budapest University of Technology and Economics, Műegyetem rkp. 5., H-1111 Budapest, Hungary*

Abstract

The construction of stability charts of linear autonomous time-delay systems is revisited in the frequency domain. The number of unstable characteristic roots is obtained from the argument principle by tracking the phase of the characteristic function along the imaginary axis; the winding integral is reformulated as an ordinary differential equation and solved by an adaptive high-order integrator, with the integrand evaluated exactly by forward-mode automatic differentiation. The adaptive frequency march makes practically unbounded integration ranges affordable, and the result is self-validating: the distance of the computed winding number from the nearest integer is a dimensionless error measure that requires no problem-specific tuning and that scales with the requested tolerance, so the accuracy of the count can be prescribed in advance; the one failure mode this residual cannot see – a root peak skipped immediately at a stability boundary – is exposed by a built-in cross-check against the independently tracked root. During the same sweep, the characteristic root closest to the integration line – typically the rightmost root – is estimated as a computational by-product and is subsequently refined by higher-order corrections built on the same automatic derivatives. This root estimate turns the otherwise piecewise-constant root-counting map into an interpolable field, so a single chart simultaneously displays the number of unstable roots and the spectral gap of the stable domain. The true stability boundary is traced at high resolution by the multi-dimensional bisection method driven by the estimated root distance as a scalar objective, and the most robust parameter point is located as the center of the largest circle inscribed in the stable domain. The characteristic function is extracted algorithmically from a simulation-ready right-hand side, including singular mass matrices (delay differential–algebraic systems). Retarded, neutral, distributed-delay, transcendental, high-dimensional and fractional-order case studies demonstrate accurate chart generation at interactive rates (complete charts in fractions of a second on a desktop machine, after a one-time startup cost), and comparisons with adaptive quadrature and semi-discretization quantify the efficiency of the approach. The method is available in the open-source Julia package `InterpolatedNyquist.jl`.

Keywords: time-delay systems, stability chart, Nyquist criterion, argument principle, neutral delay differential equation, differential–algebraic equation, automatic differentiation, adaptive integration

1. Introduction

Time delays are inherent in a wide range of engineering systems: regenerative machine-tool vibrations [1, 2, 3, 4], digitally controlled and teleoperated machines, active vibration suppression, wheel shimmy, and traffic or population dynamics all lead to delay differential equations (DDEs) [5, 6, 7, 8]. Since the characteristic function $\mathcal{D}(\lambda)$ of even the simplest linear autonomous DDE is transcendental, it possesses infinitely many characteristic roots, and the stability of an equilibrium can rarely be decided in closed form. Engineering practice therefore relies on *stability charts*: maps over a plane of design parameters that show where the equilibrium is asymptotically stable [9, 10]. The practical value of such a chart depends on two properties of the underlying numerical method: how fast a single parameter point can be classified, and how much extra information (dominant root location, spectral gap, robustness margin) the classification yields.

A large family of methods operates in the time domain by discretizing either the solution operator or the infinitesimal generator of the delay system. The semi-discretization method of Insperger and Stépán [11, 12, 13, 9], the full-discretization method [14], Chebyshev collocation [15], pseudospectral differencing [16, 17], and the spectral element method [18] all construct a finite transition matrix whose eigenvalues approximate the characteristic multipliers or exponents; spectral methods to certify all roots in a prescribed right half-plane are also available [19]. Mature software packages implement these ideas: DDE-BIFTOOL [20, 21], TRACE-DDE [22], Knut [23, 24], and `SemiDiscretizationMethod.jl` [25], which also contains the recent multiplication-free variant with linear time complexity [26]. The great strength of this family is generality: time-periodic coefficients (e.g., milling) are handled by the same machinery through Floquet theory. For *autonomous* systems, however, this generality is paid for at every single parameter point: a large discretization matrix is built and its dominant eigenvalues are computed, although only the sign of the rightmost characteristic exponent – one bit of information – decides stability. Moreover, delayed systems of neutral type (where the essential spectrum destroys the convergence of many discretizations [27, 28]), systems with algebraic constraints, i.e., delay differential–algebraic equations (DDAEs) [29, 30], distributed delays [31], and fractional-order systems [32, 33, 34] each require dedicated, nontrivial extensions in the time domain.

The frequency domain offers a complementary route, since all the listed system classes share the same abstract object: an analytic characteristic function $\mathcal{D}(\lambda)$ whose roots in the right half-plane must be counted (for a comprehensive treatment see [35]). Classical tools include the D-subdivision method [36], the analytical stability criteria of Stépán [6], the Nyquist and Mikhailov criteria [37, 38], the direct method of Walton and Marshall [39], the Rekasius substitution [40], the cluster treatment of characteristic roots [41], and the frequency-sweeping tests of Li, Niculescu and Çela [42]; the Lambert- W formalism [43] and the matrix method of Louisell [44] target the dominant roots directly, the quasi-polynomial root finder QPmR [45, 46] computes all roots in a prescribed rectangle from the values of $\mathcal{D}(\lambda)$ on a grid – a contour-integral strategy also developed for general nonlinear eigenvalue problems by Beyn [47] – and the YALTA toolbox [48] automates root-locus and stability analysis for classical and fractional delay systems. A recent survey of argument-principle-based counting techniques is given in [49].

Two frequency-domain strategies are particularly relevant here. The first solves the two real equations $\text{Re } \mathcal{D}(i\omega) = \text{Im } \mathcal{D}(i\omega) = 0$ in the parameter space extended by the frequency ω , which yields the D-curves, i.e., the parameter combinations where a root crosses the imaginary axis. The multi-dimensional bisection method (MDBM) [50, 51] traces these one-dimensional solution manifolds very efficiently. The D-curves, however, are *not* the stability boundaries: they merely mark root crossings, and the number of unstable roots on either side remains unknown. If the system is well understood, the stable side can be picked by inspection or by a few test computations, but automating this decision is surprisingly hard: the curves must be assembled into closed cells by triangulation, numerical noise generates many spurious slivers that must be filtered, and the bookkeeping often costs more than the curve tracing itself. The turning stability-lobe diagram illustrates a second, more fundamental inefficiency: towards low spindle speeds the lobe structure becomes infinitely dense, and almost all D-curves lie deep inside the unstable domain (cf. Appendix A) [52, 53, 54], so the method spends most of its effort on curves that never bound the stable region.

The second strategy counts the unstable roots directly by the argument principle, i.e., by the winding number of $\mathcal{D}(i\omega)$ – the generalized Nyquist, or Mikhailov, criterion [6, 55]. Root counting by contour integrals of $\mathcal{D}'/\mathcal{D}$ has a long history for general analytic functions [56, 57, 58]; for delay systems, Hassard [59] derived a semi-analytical counting formula, Zítek and Vyhřídál formulated argument-increment criteria for neutral systems [60, 61], and, closest to the present work, Xu and Wang [62] and Xu, Stépán and Wang [63] evaluated the argument-principle “testing integral” numerically, observing that a rough quadrature suffices because the result must be an integer. The appeal of root counting is that the answer $Z(p)$ is exactly the quantity a stability chart needs. Its curse is twofold. First, $Z(p)$ is a piecewise-constant integer field: it jumps abruptly across the stability boundaries, so it cannot be interpolated, and brute-force charts need fine parameter grids. Second, the numerical evaluation is hardest *exactly at the stability boundary* – the only region one actually cares about – because as a root approaches the integration line the integrand of the winding integral develops a near-singular peak whose width shrinks to zero, and fixed grids or rough quadratures either miss the peak or need excessive refinement; the infinite frequency range poses an additional truncation problem.

This paper revisits the integration-based Nyquist criterion and turns it into a fast and robust tool with *interpolable* output. The contributions are the following. (i) The winding integral is reformulated as a scalar ordinary differential equation for the phase, marched by a high-order adaptive integrator [64, 65, 66] whose step-size control resolves the near-singular peaks and crosses the practically unbounded frequency range, with the integrand evaluated exactly by forward-mode automatic differentiation [67]. (ii) The location of the characteristic root closest to the integration line – normally the rightmost root – is estimated *during the same sweep* at negligible extra cost, and can be refined to machine precision by a few Newton corrections for a few percent of the cost; tracking several local minima yields multiple roots at once. This ordered march, and not accuracy or speed, is what distinguishes the phase ODE from plain quadrature: a recursively subdividing quadrature visits the frequencies out of order and cannot recognize the minima at all. (iii) The root estimate makes the chart interpolable: a combined coloring shows the integer number of unstable roots outside and the smooth spectral gap inside the stable domain, which is a strikingly effective visual diagnostic. (iv) The integrality of the computed winding number provides a built-in, dimensionless error estimate

that scales with the requested solver tolerance and lets the accuracy of the count be prescribed in advance; where it is structurally blind – a skipped peak costs exactly $\pm\pi$ and leaves the result an integer – the independently obtained root supplies the missing check. (v) The estimated root distance serves as a scalar objective for MDBM, which then traces the *true* stability boundary (not the D-curves) at very high equivalent resolution, and the most robust parameter point is found as the center of the largest circle inscribed in the stable domain. (vi) The characteristic function is extracted algorithmically from a simulation-ready right-hand side – including singular mass matrices (DDAEs) – so the user never has to derive $\mathcal{D}(\lambda)$ by hand. (vii) The method applies verbatim to retarded, neutral, distributed-delay, transcendental, high-dimensional and (preliminarily) fractional-order systems, and is released as the open-source Julia [68] package `InterpolatedNyquist.jl`. Compared with the rough-quadrature testing integral of [62, 63], the present formulation adds adaptive error control tied to the integer-residual estimate, the rightmost-root by-product that enables interpolation and boundary tracing, and a far wider class of demonstrated applications; compared with contour-moment methods such as ZEAL [58], it needs no bounded search region and is tailored to the half-line stability sweep.

The paper is organized as follows. Section 2 summarizes the system classes and the argument-principle counting formula. Section 3 presents the phase ODE, the rightmost-root tracking, the higher-order refinement, and the self-validating error estimate. Section 4 assembles these ingredients into fast stability charts. Section 5 describes the automatic extraction of $\mathcal{D}(\lambda)$ from a time-domain model. Section 6 walks through the complete workflow on a constrained two-degree-of-freedom system with delayed control, Section 7 quantifies accuracy, CPU time and memory, including a comparison with the semi-discretization method, and Section 8 concludes. Appendix A collects twelve further case studies from all supported system classes.

2. Linear autonomous delay systems and the argument principle

2.1. System classes and the characteristic function

Consider the linear autonomous delay system in first-order descriptor form

$$\mathbf{E} \dot{\mathbf{x}}(t) = \mathbf{A}_0 \mathbf{x}(t) + \sum_{j=1}^m (\mathbf{A}_j \mathbf{x}(t - \tau_j) + \mathbf{B}_j \dot{\mathbf{x}}(t - \tau_j)) + \int_0^{\theta_{\max}} \mathbf{W}(\vartheta) \mathbf{x}(t - \vartheta) d\vartheta, \quad (1)$$

with state $\mathbf{x}(t) \in \mathbb{R}^N$, constant matrices $\mathbf{E}, \mathbf{A}_j, \mathbf{B}_j \in \mathbb{R}^{N \times N}$, discrete delays $\tau_j > 0$ and a distributed-delay kernel $\mathbf{W}$. The exponential ansatz $\mathbf{x}(t) = \xi e^{\lambda t}$ leads to the characteristic matrix and the characteristic function

$$\mathbf{M}(\lambda) = \lambda \mathbf{E} - \mathbf{A}_0 - \sum_{j=1}^m (\mathbf{A}_j + \lambda \mathbf{B}_j) e^{-\lambda \tau_j} - \int_0^{\theta_{\max}} \mathbf{W}(\vartheta) e^{-\lambda \vartheta} d\vartheta, \quad \mathcal{D}(\lambda) = \det \mathbf{M}(\lambda). \quad (2)$$

Equation (1) covers all the system classes treated in this paper: *retarded* DDEs ($\mathbf{E} = \mathbf{I}, \mathbf{B}_j = \mathbf{0}$), *neutral* DDEs ($\mathbf{B}_j \neq \mathbf{0}$), *delay differential–algebraic* equations (DDAEs, singular $\mathbf{E}$, e.g., due to kinematic constraints or algebraic force balances), and *distributed* delays. Beyond (1), the method below requires only an analytic function $\mathcal{D}(\lambda)$ that

can be evaluated at complex arguments; hence transcendental characteristic functions of continua (e.g., cosh-type expressions of elastic beams [69]) and fractional-order quasi-polynomials containing terms λ^α , $\alpha \notin \mathbb{N}$, are admissible as well.

The equilibrium of (1) is asymptotically stable if all roots of $\mathcal{D}(\lambda) = 0$ satisfy $\text{Re } \lambda < 0$. More generally, a guaranteed decay rate (spectral gap) $|\sigma|$ – often called γ -stability – requires all roots to lie left of the shifted line $\text{Re } \lambda = \sigma < 0$. We denote by $Z(\mathbf{p})$ the number of roots with $\text{Re } \lambda > \sigma$ for the design parameters $\mathbf{p}$; the stability chart is the map $\mathbf{p} \mapsto Z(\mathbf{p})$, and the stable (or γ -stable) domain is the region $Z = 0$.

For large $|\lambda|$ the characteristic function grows like $\mathcal{D}(\lambda) \sim c \lambda^n$, where the leading order n equals N for retarded systems in state-space form but is smaller for DDAEs (the algebraic rows remove dynamic content) and is only an *effective, possibly non-integer* order for neutral ($\mathcal{D} \sim \lambda^n(1 + \sum_j b_j e^{-\lambda \tau_j})$) and fractional systems. The counting formula below needs n , and Section 3.1 shows that it can be estimated numerically without any structural knowledge.

Fractional orders and the branch cut. Fractional powers deserve an explicit word here, because they are the one class in which the choice of integration line is a question of validity rather than of cost. The principal branch of λ^μ is analytic on the complex plane cut along the negative real axis, and $\mathcal{D}$ inherits that cut: for the fractional controller of Appendix A the two sides of the cut differ by $|\mathcal{D}(-0.25 + i0^+) - \mathcal{D}(-0.25 - i0^-)| = 14.9$, so $\mathcal{D}$ is *discontinuous* there, not merely non-smooth.

The cut lies on the *negative* reals, hence entirely outside the closed right half-plane, and the imaginary axis itself is untouched: the same measurement across the axis returns a jump of $0e + 00$ – identically zero – at every frequency tested. The choice $\sigma = 0$ is therefore admissible, and this is not a technicality about one example: it is what makes the whole class available to the method. Two provisos come with it. First, the branch *point* at $\lambda = 0$ lies *on* the contour, where $\mathcal{D}' \sim \mu \lambda^{\mu-1}$ diverges for $\mu < 1$; the resulting integrand singularity $\sim \omega^{\mu-1}$ is integrable for $\mu > 0$, and the standard indentation of the contour around the origin contributes nothing in the limit *provided* $\mathcal{D}(0) \neq 0$. Second, when $\mathcal{D}(0) = 0$ the origin is itself a characteristic root sitting on the contour and no count is defined – for the fractional PI controller $\mathcal{D}(0) = k_i$, so the line $k_i = 0$ crossing that chart is exactly such a degeneracy, and the package reports it through the invalid marker of Section 3.4 rather than by inventing a number. That the construction then works is checkable rather than assumed: over 1800 points of the two fractional charts the integer residual has median $3.8 \cdot 10^{-3}$ and never exceeds 0.010.

A *shifted* line ($\sigma < 0$) is a different matter entirely: it encloses the segment $[\sigma, 0]$ of the cut, $\mathcal{D}$ is not analytic inside the contour, and the argument principle does not apply. Appendix A demonstrates what that costs.

2.2. Counting unstable roots on a half-line

Let the integration line $\lambda = \sigma + i\omega$ be free of characteristic roots, and let $\mathcal{D}$ be analytic with finitely many roots to the right of the line. Applying the argument principle to the D-shaped contour composed of the line and the arc at infinity, and exploiting the conjugate symmetry $\mathcal{D}(\bar{\lambda}) = \overline{\mathcal{D}(\lambda)}$ of real-coefficient systems, the number of roots to the

right of the line is obtained from frequencies $\omega \geq 0$ only [6, 59]:

$$Z = \frac{n}{2} - \frac{1}{\pi} \Delta\theta, \quad \Delta\theta = \int_0^\infty \theta'(\omega) d\omega, \quad \theta(\omega) = \arg \mathcal{D}(\sigma + i\omega), \quad (3)$$

where θ is continuous in ω and $n/2$ accounts for the phase gathered on the arc at infinity, where $\mathcal{D} \sim c\lambda^n$. This is the half-line (Mikhailov-type) form of the Nyquist criterion. Writing $\mathcal{D}'(\omega) \equiv \partial_\omega \mathcal{D}(\sigma + i\omega)$ for the derivative along the line, the phase slope is

$$\theta'(\omega) = \text{Im} \frac{\mathcal{D}'(\omega)}{\mathcal{D}(\sigma + i\omega)} = \frac{\text{Re } \mathcal{D} \cdot \partial_\omega \text{Im } \mathcal{D} - \text{Im } \mathcal{D} \cdot \partial_\omega \text{Re } \mathcal{D}}{|\mathcal{D}|^2}. \quad (4)$$

135 For $\sigma = 0$, Z counts the unstable roots; for $\sigma < 0$ it counts the roots violating the prescribed spectral gap, so the same formula draws γ -stability charts.

Three standing assumptions deserve explicit statement. (i) *Finitely many roots right of the line, none on it.* For retarded systems and regular DDAEs of low index this holds for every σ greater than some abscissa; for DDAEs of higher index the spectrum can acquire advanced-type root chains that run into the right half-plane, and the count is then infinite for any line [70, 30] – such systems are outside the scope of (3), although the diagnostic of Section 3.4 flags them, because the truncated integral then fails to settle to an integer. (ii) *Neutral tail.* For neutral systems $\mathcal{D} \sim c\lambda^n(1 + \sum_j b_j e^{-\lambda\tau_j})$ and the phase of the parenthesis oscillates without decay as $\omega \rightarrow \infty$: the integral in (3) does not converge but oscillates within a bounded envelope. If the neutral amplitude satisfies $\beta = \sum_j |b_j| < 1$ – the strong-stability condition of the difference operator [27, 28] – that envelope is bounded by $\arcsin(\beta)/\pi < 1/2$ per delay term, so truncating at a generic $\omega_{\max}$ and rounding still recovers the exact count. As $\beta \rightarrow 1$ (essential instability approaching) the bound saturates and the integer residual of Section 3.4 grows accordingly – the diagnostic degrades exactly where the answer itself becomes fragile, which is the honest behaviour (see the acceleration-feedback case study A.6 in Appendix Appendix A). (iii) *Analyticity right of the line.* Fractional powers λ^α are taken on the principal branch, whose cut lies on the negative real axis; for $\sigma \geq 0$ the region right of the line is then free of branch points, and (3) applies verbatim. A shifted line with $\sigma < 0$, however, crosses the cut, and the count refers to the roots of the principal-branch extension; case study A.11 in Appendix Appendix A discusses the practical consequences.

2.3. Why the naive evaluation is impractical

Formula (3) looks like a textbook quadrature problem, but three properties make its direct use for stability charts inefficient. First, $Z(\mathbf{p})$ is integer-valued: it is constant inside every cell of the chart and jumps at the boundaries, so the map cannot be interpolated, and any grid-based chart must be fine enough to resolve every cell of interest. Second, the numerical evaluation is hardest exactly where it matters most. If a root $\lambda_r = \sigma_r + i\omega_r$ lies close to the integration line, the integrand (4) develops a near-Lorentzian peak at $\omega \approx \omega_r$,

$$\theta'(\omega) \approx \frac{-(\sigma_r - \sigma)}{(\sigma_r - \sigma)^2 + (\omega - \omega_r)^2} + \text{smooth part}, \quad (5)$$

whose half-width $|\sigma_r - \sigma|$ tends to zero as the parameter point approaches the stability boundary, while its integral contribution stays $\pm\pi$; any method that samples the integrand pointwise and happens to place all its samples outside

such a peak miscounts Z by exactly one, and – since the result stays an integer – does so without any warning. Third, the integration range is infinite: the phase slope decays only algebraically, with delay-induced oscillatory ripples, so a fixed truncation $\omega_{\max}$ must balance truncation error against the cost of resolving a ripple over a huge interval.

Section 3 addresses the first and third difficulties with the step-size control of an adaptive integrator, and turns the second into an asset by extracting the location of the offending root from the very peak that makes the integration hard. The peak-skipping hazard, by contrast, cannot be legislated away by any choice of integrator; what can be done – and what Section 3.4 does – is to detect it, using the fact that the root estimate and the phase count come from independent mechanisms and must agree.

3. Ordered adaptive integration with rightmost-root tracking

3.1. The winding integral as an ordinary differential equation

The key step is disarmingly simple: instead of applying quadrature to (3), the accumulated phase is treated as the state of the scalar initial value problem

$$y'(\omega) = \theta'(\omega) = \operatorname{Im} \frac{\mathcal{D}'(\omega)}{\mathcal{D}(\sigma + i\omega)}, \quad y(0) = 0, \quad \tilde{Z} = \frac{n}{2} - \frac{y(\omega_{\max})}{\pi}, \quad Z = \operatorname{round}(\tilde{Z}), \quad (6)$$

which is handed to a modern adaptive integrator with local error control. It should be stated plainly what this formulation is and is not. The right-hand side of (6) does not depend on the state y , so the Jacobian $\partial_y y'$ vanishes identically: (6) is a quadrature problem in disguise, it is never stiff in the classical sense, and an automatically stiffness-switching integrator provably never switches on it. What the integrator contributes is therefore not stiffness handling but *adaptive step-size control on an ordered march*: steps contract around the near-singular peaks (5), where the phase turns by $\pm\pi$ over a frequency interval of width $|\sigma_r - \sigma|$, and expand again over the smooth stretches. Because the integrand is analytic, a high-order explicit Runge–Kutta pair is the appropriate choice; the default is Vern9 [64], which at loose tolerances is three orders of magnitude more accurate than the fifth-order alternative [65] and at tight tolerances is also faster (Section 7.1). The ordered march is what makes the root tracking of Section 3.2 possible, and that – not stiffness – is the reason the phase ODE is preferred here over plain quadrature.

The integrand is evaluated *exactly*: the characteristic function is called once per frequency with a dual-number argument $\lambda = \sigma + i(\omega + \delta)$, $\delta^2 = 0$ (distinct from the integer residual ε of Section 3.4), so forward-mode automatic differentiation [67] delivers $\mathcal{D}$ and $\mathcal{D}' = \partial_\omega \mathcal{D}$ simultaneously, without finite differencing and without any user-supplied derivative. A tiny offset $\omega \mapsto \max(\omega, 10^{-9})$ regularizes branch points of fractional terms at the origin.

Two practical parameters remain. The truncation frequency $\omega_{\max}$ can be chosen generously: past the highest structural resonance the phase slope decays algebraically, the adaptive step size grows accordingly, and a *too small* $\omega_{\max}$ is immediately visible in the result, because the truncated tail shifts $\tilde{Z}$ away from an integer (Section 3.4). The cost of the tail is governed by the decay of the delay-induced phase ripple: if the delayed terms sit at least two orders below the leading power of $\mathcal{D}$ – as for delayed *position* feedback on mechanical systems – the ripple decays at least

like ω^{-2} , and pushing $\omega_{\max}$ far beyond the resonances is essentially free: for the fourth-order benchmark the march costs 1426, 1474 and 1506 evaluations of $\mathcal{D}$ at $\omega_{\max} = 10^6, 10^8$ and 10^{10} respectively – four extra decades of frequency for six percent more work, with $\tilde{Z}$ unchanged to seven digits. One caution applies to those giant steps: an adaptive error estimator only *samples* the local error, so a step that grows to many ripple periods can in principle alias a slowly
 195 decaying oscillation; in practice the embedded error estimate of a high-order pair rejects such steps long before the count is affected, and any surviving aliasing is exposed by the integer residual of Section 3.4. If a delayed term sits only one order below the leading power (delayed *velocity* feedback), the ripple decays merely like ω^{-1} , the solver keeps resolving it (Fig. 2c), and every resolved step contributes its share of local error; there $\omega_{\max}$ does cost, and the integer-residual indicator of Section 3.4 exposes both a truncated tail and an excessively long march at a glance.

200 The resulting recommendation is asymmetric, and deliberately so. *For accuracy, be generous*: $\omega_{\max} = 10^6$ is a safe default for every system in this paper, and 10^8 costs almost nothing extra where the ripple decays quickly. *For speed, be frugal*: two to three decades above the highest resonance ($\omega_{\max} = 10^4$ for the mechanical systems here) leaves a truncated tail of order $0.2/\omega_{\max}$ in $\tilde{Z}$ – five orders of magnitude below the rounding threshold – and, for a slowly decaying ripple, cuts the evaluation count by more than an order of magnitude. The charts of Section 6 exploit
 205 the first setting; the gallery of Appendix Appendix A exploits the second. Since a too-small $\omega_{\max}$ announces itself in the residual, the choice can be made empirically rather than by analysis.

The leading order n in (6) is estimated numerically, and the natural place to do so is not the imaginary axis at all. The term $n/2$ accounts for the phase gathered on the arc at infinity, and that arc lies in the *right half-plane*, where every delayed factor $e^{-\lambda\tau_j}$ decays exponentially. Probing along the positive real axis therefore measures precisely the
 210 quantity the counting formula needs, with the delay terms already extinguished:

$$n \approx \text{LS-slope} \left(\ln s_k \mapsto \ln |\mathcal{D}(s_k)| \right), \quad s_* \gg 1, \quad (7)$$

$s_k \in [s_*/10, s_*]$

the least-squares slope of the log–log magnitude over a handful of log-spaced real samples. Two points about s_* deserve emphasis. It is *decoupled from the integration range*: n is a property of $\mathcal{D}$ alone, so the probe may sit far beyond $\omega_{\max}$ – the default is $s_* = 10^8$ whatever $\omega_{\max}$ is – and it costs a handful of evaluations once per sweep, not once per frequency. And on the real axis a larger s_* genuinely buys accuracy: $|\mathcal{D}(s)| \rightarrow |c| s^n$ with a smooth algebraic
 215 correction, so the estimate converges as $O(1/s_*)$, measured here as $7.5 \cdot 10^{-5}$, $7.4 \cdot 10^{-7}$ and $7.4 \cdot 10^{-9}$ at $s_* = 10^4, 10^6$ and 10^8 – including the effective integer order of neutral equations and the non-integer order of fractional ones.

That last point is exactly what an imaginary-axis fit cannot offer, and the contrast explains why the difficulty is worth side-stepping rather than fighting. On the imaginary axis $|e^{-i\omega\tau}| = 1$, so the delay terms superimpose a ripple on $\ln |\mathcal{D}(i\omega)|$ that a log-spaced fit aliases rather than averages; for a neutral system, where the ripple does not decay
 220 at all, *no* frequency is large enough, and a single-point evaluation of $\omega \operatorname{Re}(\mathcal{D}'/\mathcal{D})$ does not merely stall but diverges in proportion to ω (measured: $n_{\text{est}} = -226, 5132$ and $80\,028$ at $\omega = 10^3, 10^4$ and 10^6 for a system whose true effective order is 2). Even for retarded systems the same estimator retains an error of $O(b\tau/c)$ at arbitrarily large ω when a delayed term sits one order below the leading power (b the delayed-term coefficient, c the leading one).

Probing where the delays are exponentially small removes all of this at a stroke. The estimator needs no structural knowledge of the model, costs a handful of extra evaluations, and in parameter sweeps is computed once when n is parameter-independent.

Two limitations are worth naming. The first is representability: $|\mathcal{D}(s)| \sim s^n$ overflows for large n , so s_* is reduced automatically until $|\mathcal{D}|$ is finite, which for very high-order determinants caps the achievable accuracy; the leading order may then be supplied explicitly. There is, however, a better answer than fighting the symptom, and it costs nothing. Whenever the model offers a natural open-loop/feedback split – as every control problem and every regenerative-machining model does – the *return difference* $\mathcal{D} = \det(\mathbf{I} + \mathbf{Q}_0^{-1}\mathbf{F}) = 1 + L(\lambda)$ (with $\mathbf{F}$ the feedback matrix) has exactly the same zeros as $\det(\mathbf{Q}_0 + \mathbf{F})$, adds poles only at the open-loop roots (which, for an open-loop stable structure, lie in the left half-plane and leave the right-half-plane count untouched), and tends to 1 at infinity. Then $n = 0$ *exactly*, no estimate is needed, and the enormous magnitudes disappear: for the 29-DOF finite element bar of Appendix Appendix A the raw determinant grows like λ^{58} , reaches $\sim 10^{39}$ at the top of the integration range, and leaves an integer residual of 0.12 – a rounded count barely better than a coin toss – while the return difference leaves $5.0 \cdot 10^{-1}$. Where the split exists, it is worth more than any tuning of the integrator.

The second limitation is more interesting. Equation (7) presumes that $\mathcal{D}$ is polynomially bounded in the right half-plane, which holds for every quasi-polynomial $\mathcal{D} = \sum_j a_j \lambda^{k_j} e^{-\lambda \tau_j}$ with $\tau_j \geq 0$ – hence for all retarded, neutral and differential–algebraic systems – but not for the transcendental characteristic functions of continua, where $\cosh(\sqrt{\lambda^2 + c\lambda}L)$ grows like $e^{\lambda L}$ and no finite leading power exists on the real axis at all. The package detects this automatically – for a genuine power law the fitted slope must be the same one window lower, and for an exponentially growing $\mathcal{D}$ it is not – and falls back to a least-squares fit along the imaginary axis, where such functions are bounded and oscillatory and the fit returns the effective order the counting formula needs ($n \approx 0$ for the beam of Appendix Appendix A). The fallback inherits the delay ripple and is correspondingly less accurate, which is precisely why it is not the primary route.

The requested relative and absolute solver tolerances control the local error of the accumulated phase, measured in radians. Since the exact winding number is an integer multiple of π in the phase, the error of $\tilde{Z}$ inherits the tolerance directly as a *dimensionless* quantity: the same default (10^{-5}) is appropriate for a machining model and for a dense 50×50 matrix determinant alike, with no problem-dependent scaling (Section 3.4).

3.2. Rightmost-root estimation as a by-product

The peaks of the integrand are not only a numerical nuisance – they carry the information where the dominant characteristic roots are. Along the integration line, $|\mathcal{D}(\sigma + i\omega)|^2$ dips to a sharp local minimum at $\omega \approx \omega_r$ for every nearby root $\lambda_r = \sigma_r + i\omega_r$, and the smallest minimum belongs to the root closest to the line, which for $\sigma = 0$ is typically the rightmost root that decides stability. The solver therefore tracks $\min_\omega |\mathcal{D}|^2$ during the march at negligible cost: at every integrand evaluation the already computed quantities $\mathcal{D}$ and $\mathcal{D}'$ yield the real part of one Newton step

towards the root,

$$\hat{\sigma} = \sigma + \operatorname{Re}\left(-\frac{\mathcal{D}}{d\mathcal{D}/d\lambda}\right) = \sigma - \frac{\operatorname{Re} \mathcal{D} \cdot \partial_{\omega} \operatorname{Im} \mathcal{D} - \operatorname{Im} \mathcal{D} \cdot \partial_{\omega} \operatorname{Re} \mathcal{D}}{|\mathcal{D}'|^2}, \quad (8)$$

using $d\mathcal{D}/d\lambda = -i\mathcal{D}'$, and the frequency of the minimum is stored as $\hat{\omega}$. The pair $(\hat{\sigma}, \hat{\omega})$ is a first-order estimate of the dominant root location returned together with Z ; its accuracy is highest exactly at the stability boundary, where the root lies on the line and the Newton step becomes exact up to the solver tolerance. Away from the boundary the estimate degrades gracefully, but there only its sign and magnitude matter – as a smooth measure of the distance to instability.

Several roots can be tracked simultaneously. A compile-time variant of the solver detects every local minimum of $|\mathcal{D}|^2$ through the sign change of its ω -derivative, $2(\operatorname{Re} \mathcal{D} \partial_{\omega} \operatorname{Re} \mathcal{D} + \operatorname{Im} \mathcal{D} \partial_{\omega} \operatorname{Im} \mathcal{D})$, and maintains the N_r best candidates in a sorted, allocation-free buffer; a boundary test at $\omega = 0$ captures real dominant roots, since $|\mathcal{D}|^2$ is even in ω . Tracking one root is the default and adds a few percent to the sweep time; tracking zero roots gives maximal sweep speed. One caveat deserves emphasis: the *deepest* minimum of $|\mathcal{D}|$ belongs to the rightmost root near the stability boundary, but deeper inside the stable or unstable domains a different root branch may produce a deeper dip. When the true dominant root is required away from the boundary, several minima should be tracked ($N_r \approx 5$ – 10), all of them refined, and the dominant root taken as the one with the maximal real part – this recipe is validated against an independent eigenvalue method in Section 7.3.

That recommendation is worth more than a footnote, because the single-minimum estimate is not merely inaccurate away from the boundary: it is *discontinuous*. Where one branch overtakes another as the closest one, $\hat{\omega}$ jumps between modes and $\hat{\sigma}$ jumps with it – on the showcase system, from -0.194 to -0.053 as $\hat{\omega}$ jumps from 0.96 to 2.34 , across a single step in the gain. Any quantity built on it inherits the jump, and a method that fits a local linear model – as the boundary tracer of Section 4.2 does inside each cell – will read a steep gradient as a zero crossing and place a spurious boundary point in the middle of the stable domain. The dominant root passes through a branch crossing smoothly, because it is a maximum over branches rather than the nearest one, so building the coloring and the boundary objective from it removes the artefact at its source. For the count itself the caveat is immaterial, since near the boundary the crossing root always produces the global minimum.

3.3. Higher-order root refinement

The tracked estimate $(\hat{\sigma}, \hat{\omega})$ is refined by local corrections built from the same automatic derivatives. Writing the Taylor expansion at $\hat{\lambda} = \hat{\sigma} + i\hat{\omega}$,

$$\mathcal{D}(\hat{\lambda} + \Delta) \approx \mathcal{D} + \mathcal{D}_{\lambda} \Delta + \frac{1}{2} \mathcal{D}_{\lambda\lambda} \Delta^2 + \frac{1}{6} \mathcal{D}_{\lambda\lambda\lambda} \Delta^3 = 0, \quad (9)$$

where the subscripts denote derivatives with respect to λ evaluated at $\hat{\lambda}$, the package offers (i) *Newton–Raphson* iterations $\Delta = -\mathcal{D}/\mathcal{D}_{\lambda}$ (four steps by default); (ii) a single *quadratic* (Poly-2) or (iii) *cubic* (Poly-3) Taylor solve, where the root of the local cubic closest to $\hat{\lambda}$ is selected via the eigenvalues of its companion matrix. The first derivative is exact (automatic differentiation); the higher derivatives are obtained by central differences of the exact

first derivative, which avoids the compilation cost of nested dual numbers while keeping truncation errors far below the achievable accuracy. An optional mode freezes $\hat{\omega}$ and corrects only the real part, which is convenient when the frequency is pinned by measurement. The accuracy and cost of the variants are quantified in Section 6.4; the practical summary is that every variant costs a few percent of the phase integration, that Newton is simultaneously the cheapest and the most accurate – hence the default – and that a few steps converge to machine precision near the boundary.¹ If several estimates are tracked, the same refinement applied to each returns a set of dominant roots, comparable to the output of dedicated root finders such as QPmR [45] – with the difference that here the root solve is a cheap post-processing step of the stability count.

3.4. Self-validating error estimate

The exact winding number is an integer, so

$$\varepsilon = |\tilde{Z} - \text{round}(\tilde{Z})| \quad (10)$$

is a rigorous, freely available indicator of the accumulated numerical error: it contains the local integration errors, the tail truncated at $\omega_{\max}$, and the residual of the estimated order (7). Since the phase is integrated in radians, ε is *dimensionless* and scales essentially linearly with the requested tolerance – amplified by the accepted-step count, since the per-step error tolerance accumulates along the march – hence the tolerance acts as a universal accuracy dial that transfers between systems without re-tuning. In the numerical experiments of Section 6.3, ε tracks the tolerance across six decades: at 10^{-2} the residual reaches order one and genuine miscounts appear – the deliberate failure demonstration of Fig. 4 – while 10^{-4} already keeps ε an order of magnitude below the rounding threshold $1/2$ and produces reliable charts at near-maximal speed, and 10^{-8} yields reference-quality values at a moderate CPU-time premium; the package default 10^{-5} adds a further safety decade at little cost.

Supplying the leading order. The order n is an *input* of the counting formula (3), not a result of it, and the estimator of Section 5 exists only so that a user who does not know n is not blocked. Whenever n is known – and it usually is, by inspection of the model – it should simply be passed (`n_power_max`), for every back-end and for a whole chart at once. It is then exact rather than estimated, and it costs nothing. This matters most exactly where the estimator is least comfortable: a neutral $\mathcal{D} = \lambda^2(1 + ae^{-\lambda}) + \dots$ has $n = 2$ by inspection, because $e^{-s} \rightarrow 0$ along the real axis, yet on the imaginary axis $|\mathcal{D}|$ never settles onto a power law – it oscillates by a factor $(1 + |a|)/(1 - |a|)$, up to 39 at $|a| = 0.95$, for ever. The estimator does recover $n = 2$ there (to all printed digits, on every neutral case study of Appendix Appendix A), but the user has no way to know that in advance, and one declared number removes the doubt. The same applies to fractional orders, which are legitimate non-integer inputs.

¹Like any local root iteration, neither correction is guaranteed to converge: a Newton step can leave the basin of the intended root, a Taylor root can be spurious when the seed is poor, and at a defective (multiple) root, where $\mathcal{D}_1 \rightarrow 0$, Newton’s convergence degrades from quadratic to linear. The implementation therefore rejects runaway steps, which suffices in every case study of this paper; should a chart nevertheless show isolated irregular root estimates, refinement can be switched off (`refinement_method=:Linear`) to fall back to the raw tracked estimate, whose sign – the quantity the charts rely on – is unaffected.

When the residual stops being a check.. The diagnostic has a precondition that deserves to be stated rather than discovered, and – as the neutral case shows – it is neither the tolerance nor the leading order, but the *convergence of the tail*. Equation (10) measures the distance of $\tilde{Z}$ from an integer, and $\tilde{Z}$ is only *expected* to be an integer when the truncated remainder $\int_{\omega_{\max}}^{\infty}$ is negligible. For retarded systems it is, and ε behaves as described: across the retarded case studies of Appendix Appendix A the median residual is $\sim 10^{-4}$ and essentially no grid point is uncertain. For *neutral* systems it is not. There the integrand does not decay at all, the tail contributes an oscillation bounded by $\arcsin|a|/\pi$ rather than a vanishing remainder (Section 2.1), and the residual inherits it: on the neutral panels of Appendix Appendix A the median residual is ≈ 0.1 and between 10% and 27% of grid points exceed the 1/4 mark – with the counts nevertheless correct. For that class, therefore, ε is *not* a usable self-check, and what protects the count is the analytic bound $\arcsin|a|/\pi < 1/2$ instead. Enlarging $\omega_{\max}$ does not help and should not be attempted: the oscillation is bounded, not decaying.

One caveat is inherited from the argument principle itself and deserves to be stated sharply, because it bounds what ε can certify. A march that steps *over* a peak loses exactly $\pm\pi$ of phase, so $\tilde{Z}$ shifts by exactly one and remains just as close to an integer as before: ε measures the *quadrature* quality and is structurally blind to a skipped peak. Nor is this hypothetical: approaching a stability boundary the peak half-width $|\sigma_r - \sigma|$ collapses to many orders of magnitude below any practical step size (quantified in Table 5), and the marching solver then strides past it and reports success. No pointwise method is immune: quadrature rules that happen to place all their nodes outside such a spike are equally blind, and only survive here because the delay ripple forces them to subdivide finely everywhere anyway (Section 7.1).

What rescues the situation is the second output of the same sweep. The tracked root of Section 3.2 is obtained from the minima of $|D|^2$, a mechanism entirely independent of the accumulated phase, and it remains accurate exactly where the peak is narrow – that is, exactly where the count is fragile. The two therefore cross-validate each other: a skipped peak shows up as an *inconsistency* between $\text{sign}(\hat{\sigma} - \sigma)$ and the predicate $Z = 0$; both quantities are standard outputs of every sweep, so the check costs one comparison.

The test must, however, be restricted to a neighbourhood of the line, and the reason is instructive. Far from the boundary the tracked minimum need not belong to the *dominant* root (Section 3.2): deep inside an unstable domain the roots that make $Z > 0$ may sit far to the right and leave no dip near the line at all, so the tracked root is a stable one and disagrees with $Z > 0$ entirely legitimately. Applied blindly, the test therefore fires on 719 of the 8000 points of a *fully converged* chart of the showcase system, which would make it worthless. A peak can only be stepped over where a root lies close to the line, so the informative test is

$$\text{sign}(\hat{\sigma} - \sigma) \text{ disagrees with } [Z = 0] \quad \text{and} \quad |\hat{\sigma} - \sigma| < h, \quad (11)$$

with h a small fraction of the spread of $\hat{\sigma}$ over the chart. With $h = 0.001$ – orders of magnitude below the median $|\hat{\sigma}| \approx 0.048$ of those spurious disagreements – the test fires on only 2 of the same 8000 points, and it catches 8 of the 57 miscounts that a deliberately loose run ($\text{tol} = 10^{-2}$) produces in the only place that matters (Section 6.3). That last

qualification is the point: skipping one peak only ever changes Z by one, so the stable/unstable classification a chart displays is affected only where the last unstable root crosses – and that is precisely the regime in which $\hat{\sigma}$ is small and the test is sharp. The same mechanism that flags a skipped peak can also mend it: the minimum of $|\mathcal{D}|^2$ *locates* the peak precisely however narrow it is, and Section 6.3 shows how an optional callback uses that location to repair the count within the march itself.

3.5. Alternative integrators within the same framework

The phase integrand (4) can also be fed to classical adaptive quadrature; the package provides a Gauss–Kronrod back-end (QuadGK.jl [71]) with the identical interface. For the root count alone it is fully competitive: it stays within a factor of about three of the march in evaluations either way – cheaper when the phase ripple decays quickly, because a quadrature rule can recognize a negligible tail and stop where a march must still walk to $\omega_{\max}$, dearer when the ripple persists (Table 10) – and close to a stability boundary it is the more dependable of the two (Section 7.1). It is a reasonable choice whenever only Z is wanted. The decisive difference concerns the rightmost root. Adaptive quadrature subdivides its interval recursively and therefore visits the frequencies in an essentially arbitrary order: there is no sweep along which a local minimum of $|\mathcal{D}|^2$ could be recognized, and the bookkeeping that the ODE march gets for free would require a separate minimization. It is this ordered traversal – not accuracy, not speed, and certainly not stiffness – that earns the phase ODE its place as the default, because every chart in this paper is built from the tracked root: the interpolable coloring (12), the boundary objective (13), the refinement seeds, and the consistency check of Section 3.4 all consume $\hat{\sigma}$, not Z alone. A third back-end evaluates the integral by a fixed-step trapezoidal rule with no adaptivity and essentially zero overhead. It is included for completeness and as a real-time floor, but the measurements of Section 7.4 do not support recommending it for the systems studied here, and it is worth saying why: a *uniform* frequency grid cannot resolve the peak (5) of a root that lies close to the line, and near a stability boundary that peak is arbitrarily narrow. Its mean count error therefore converges with the step count while its *worst* error over a chart does not – at $3 \cdot 10^4$ steps, more than an order of magnitude slower than the adaptive default, individual points are still off by several roots. Adaptivity is thus not a convenience here but the very mechanism that makes the count trustworthy, which is the same reason the step-size control was introduced in Section 3.1.

4. Fast stability charts

4.1. Brute-force sweep with interpolable coloring

The single-point solve of Section 3 is embarrassingly parallel over the parameter plane; a multithreaded sweep over a grid $\mathbf{p}_{ij}$ returns the fields Z , $\tilde{Z}$, $\min |\mathcal{D}|$, $\hat{\sigma}$ and $\hat{\omega}$. The integer field Z alone is a poor plotting quantity – flat plateaus separated by jumps – but the estimated root location repairs precisely this deficiency. The combined field

$$C(\mathbf{p}) = \begin{cases} \hat{\sigma}(\mathbf{p}), & \text{if } Z(\mathbf{p}) = 0, \\ Z(\mathbf{p}), & \text{otherwise,} \end{cases} \quad (12)$$

is a remarkably efficient coloring trick for visual inspection: inside the stable domain ($Z = 0, \hat{\sigma} < 0$) it varies *smoothly* and displays the spectral gap, i.e., how far the dominant root lies from the imaginary axis, while outside it shows the integer number of unstable roots as discrete color levels. One glance at a single map thus reveals where the system is stable, how robustly stable it is there, and how many roots must be stabilized elsewhere. Because C is smooth over the stable domain, interpolation between grid points is meaningful exactly where it is useful, so a coarse and cheap grid suffices for the colored background; only the integer levels of the unstable side are plotted as flat colors.

4.2. High-resolution boundary tracing with MDBM

The stability boundary itself deserves better than pixel resolution. The multi-dimensional bisection method (MDBM) [50, 51] locates the zero set of a scalar function over a coarse initial mesh and refines only the bracketing cells, halving the mesh size in every iteration. Here *only the estimated root distance is used* as the scalar objective:

$$g(\mathbf{p}) = \text{sign}([Z(\mathbf{p}) = 0] - \tfrac{1}{2}) |\hat{\sigma}(\mathbf{p}) - \sigma|, \quad (13)$$

which is positive inside the stable (γ -stable) domain, negative outside, crosses zero exactly on the stability boundary, and – unlike Z itself – varies almost linearly across it, because near the boundary $\hat{\sigma}$ is an accurate estimate of the dominant root’s real part (Section 3.2). MDBM can therefore interpolate the zero crossing within each bracketing cell to sub-cell accuracy. One detail is essential rather than incidental: $\hat{\sigma}$ here must be the *dominant* root of Section 3.2 (several tracked minima, maximal real part), not the single closest minimum. The latter jumps where one branch overtakes another, and a linear model fitted inside a cell straddling that jump interpolates a zero that does not exist – a phantom boundary point in the interior of the stable domain, which is invisible in the traced curve but corrupts anything built on the point cloud, such as the inscribed circle of Section 4.3. Note the contrast with applying MDBM to the pair $\text{Re } \mathcal{D} = \text{Im } \mathcal{D} = 0$ in the parameter–frequency space: that approach traces *all* root-crossing D-curves, most of which lie deep in the unstable region (Section 1), while the objective (13) follows only the *true* stability limit. By the very nature of the bisection refinement, the computational effort concentrates exclusively on the boundary: k refinement levels raise the equivalent resolution of an $n_0 \times n_0$ initial mesh to $2^k n_0 \times 2^k n_0$, while the number of function evaluations grows only in proportion to the boundary length. On the fourth-order benchmark of Section 7, four refinement levels from a 20×20 mesh trace the boundary at a 320×320 -equivalent resolution from about 3500 evaluations (Table 8) – some 30 times fewer than the 10^5 a brute-force grid of that density would need.

4.3. The most robust parameter point

With a high-resolution boundary and a smooth interior metric available, the natural design question – *which parameter point is the most robustly stable?* – reduces to a geometric one: the center of the largest circle inscribed in the stable domain maximizes the distance to the stability boundary in parameter space. The package computes it from the MDBM boundary point cloud (or from the binary stable mask of the brute-force grid) by a coarse-to-fine search followed by an omnidirectional pattern refinement with adaptive step control. Anisotropic parameter scaling

410 turns the circle into an ellipse, which avoids degenerate results in elongated stable regions and lets the user weight the expected perturbations of the individual parameters. The exact radius is rarely of engineering interest; the *location* of the optimum is, and that location is insensitive to the residual discretization of the boundary. If a spectral-gap-optimal (rather than parameter-robust) point is preferred, the interpolable interior field $\hat{\sigma}(\mathbf{p})$ of (12) can be minimized instead – both options are cheap post-processing steps on data already computed.

415 4.4. σ -contours: decay-rate maps

Repeating the boundary tracing of Section 4.2 for a family of shifted lines $\sigma \in \{0, \sigma_1, \sigma_2, \dots\}$ produces the level curves of the guaranteed decay rate, i.e., a γ -stability map: each contour bounds the parameter region whose rightmost root lies left of $\text{Re } \lambda = \sigma$. The interior coloring by $\hat{\sigma}$ and the traced σ -contours are two views of the same robustness landscape – the former is a fast estimate on a coarse grid, the latter a high-resolution guaranteed boundary – and
 420 plotting them together provides a strong visual cross-validation of both (Section 6).

4.5. Recommended workflow

The complete chart-generation recipe used throughout the paper is:

1. *Coarse colored sweep.* Multithreaded evaluation of (6) on a coarse grid; plot the combined field (12). Cost: a fraction of a second to a few seconds.
- 425 2. *Boundary tracing.* MDBM on the scalar objective (13), typically 4 refinement levels from a 20×20 – 30×30 initial mesh. Cost: comparable to or below the sweep.
3. *Robust point.* Largest inscribed circle/ellipse from the MDBM point cloud. Cost: milliseconds.
4. *Optional.* σ -contour family; root refinement at selected points; higher tolerance for publication-quality boundaries.

430 Every step reuses the same single-point solver, so the entire chart refreshes fast enough for interactive parameter exploration (Section 7).

5. Automatic extraction of the characteristic function from a simulation model

In engineering practice, a system studied for stability almost always exists as a *simulation model* first: a right-hand side function written for a general-purpose integrator such as `DifferentialEquations.jl` [66]. Deriving the
 435 characteristic function (2) from it by hand is tedious and error-prone, particularly for many degrees of freedom, several delays, or algebraic constraints. The package therefore extracts $\mathcal{D}(\lambda)$ *algorithmically* from the very same right-hand side $\mathbf{f}(\mathbf{u}, \mathbf{h}, \mathbf{p}, t)$ used for time-domain simulation, where $\mathbf{h}(\mathbf{p}, t - \tau)$ is the standard history-function interface for the delayed states.

The mechanism requires a single evaluation of $\mathbf{f}$ per requested λ . The state is seeded with dual-number perturbations, $\mathbf{u} = \mathbf{u}_{\text{eq}} + \mathbf{a}$, where the components of $\mathbf{a}$ carry independent infinitesimal directions, and the history argument is replaced by the *mock exponential history*

$$\mathbf{h}(\mathbf{p}, t) = \mathbf{u}_{\text{eq}} + \mathbf{a} e^{\lambda t}. \quad (14)$$

Whatever delayed value $\mathbf{h}(\mathbf{p}, t - \tau_j)$ the model requests contributes the factor $e^{-\lambda \tau_j}$ automatically, so discrete, multiple, and (approximated) distributed delays are all captured without declaring them. Evaluating $\mathbf{f}$ at $t = 0$ and collecting the dual partial derivatives of the residual

$$\mathbf{r} = \lambda \mathbf{E} \mathbf{a} - \mathbf{f}(\mathbf{u}_{\text{eq}} + \mathbf{a}, \mathbf{h}, \mathbf{p}, 0) \quad \implies \quad \mathbf{M}(\lambda) = \frac{\partial \mathbf{r}}{\partial \mathbf{a}}, \quad \mathcal{D}(\lambda) = \det \mathbf{M}(\lambda), \quad (15)$$

yields the exact characteristic matrix of the *linearized* dynamics around $\mathbf{u}_{\text{eq}}$: the dual seeding performs the linearization, so the right-hand side may be fully *nonlinear* – no separate linearization step is required (the equilibrium itself could also be located from the same right-hand side, which is outside the scope of this paper; a residual check warns if the supplied $\mathbf{u}_{\text{eq}}$ is not an equilibrium). The mass matrix $\mathbf{E}$ is supplied separately – it is not part of the right-hand side, exactly as in the ODEFunction/DDEFunction convention of DifferentialEquations.jl – and it may be *singular*: rows of zeros in $\mathbf{E}$ turn the corresponding rows of $\mathbf{M}(\lambda)$ into the algebraic constraint equations, so delay differential–algebraic systems are handled by the identical code path. The determinant is evaluated with statically sized arrays, so a full extraction costs one right-hand-side call plus an $N \times N$ determinant, and the dual seeding composes transparently with the outer dual numbers of the phase ODE (6).

The complete user workflow is thus reduced to providing the simulation right-hand side, the mass matrix if it differs from the identity, and the parameter ranges of the chart:

```
function rhs(u, h, p, t)                # simulation model (DDE interface)
    P, D = p
    x1_del = h(p, t - 0.5; idxs=1)      # delayed position
    v1_del = h(p, t - 0.5; idxs=4)      # delayed velocity
    ...                                  # equations of motion + constraints
end

D(lam, p) = get_D_from_model(rhs, lam, p, Val(7); mass_matrix = E)
                                           # Val(7): the model has 7 states;
                                           # E is the (singular) mass matrix

Z, Zraw, minD, sigma_est, omega_est =
    calculate_unstable_roots_direct(D, (2.0, 1.0))
```

The leading order n required by the counting formula (3) is estimated automatically by (7) – including the effective orders of neutral characteristic functions and the non-integer orders of fractional ones – so no additional structural

Table 1: The price of never deriving $\mathcal{D}$ by hand. Both forms are the *same* function – they agree to machine precision and every one of the 10^4 counts is identical – and differ only in how each evaluation is obtained. Same system, same chart, same settings ($\omega_{\max} = 10^4$, default tolerance).

form of $\mathcal{D}$	chart (100×100)	per point [μ s]
hand-derived (no determinant, no AD)	1.90 s	190
automatic extraction from the RHS	38.67 s	3867
<i>ratio</i>	20×	20×

input is needed. In Section 6 the extraction is verified against a hand-derived characteristic function: the two agree to machine precision.

5.1. What the convenience costs

Automatic extraction is a convenience, and it is not free. Every evaluation of the extracted $\mathcal{D}$ runs the user’s right-hand side once with dual-number arguments and then takes a 7×7 determinant of the descriptor matrix, where the hand-derived form of the same system evaluates a flat scalar expression – a handful of multiplications, one exponential, no determinant and no automatic differentiation. Table 1 measures the difference on the identical chart: 1.90 s against 38.67 s for 100×100 points, a factor of about 20.

The trade is therefore explicit, and both sides of it are defensible. If the model exists only as a simulation right-hand side – which is the normal situation early in a design study, and the situation in which deriving $\mathcal{D}$ for a constrained multibody model is an afternoon of error-prone algebra – the extracted form gives a correct chart in a minute with no derivation whatsoever. If the same chart is to be recomputed inside an optimization loop, or refreshed interactively while a slider moves, then the algebra is worth doing once: the hand-derived form turns the same chart into a sub-second one. The recommended workflow is exactly that order – explore with the extracted form, and derive $\mathcal{D}$ by hand only once the model has settled and speed starts to matter. Everything the method does is identical in either case; only the cost of one evaluation changes. All figures in this paper use the hand-derived form, having first verified it against the extracted one (Section 6).

6. Showcase: a constrained two-degree-of-freedom system with delayed PD control

All elements of the method are now demonstrated on a single mechanical example that is simple enough to verify by hand yet exercises every nontrivial feature at once: multiple degrees of freedom, a feedback delay, and an algebraic constraint, i.e., a delay differential–algebraic system with a singular mass matrix.

6.1. Model and automatic extraction

The main structure of mass m_1 is statically unstable (inverted-pendulum-like, modeled by the negative net suspension stiffness $k_1 < 0$ and damping c_1); it carries an absorber unit of mass m_2 attached through a spring and damper

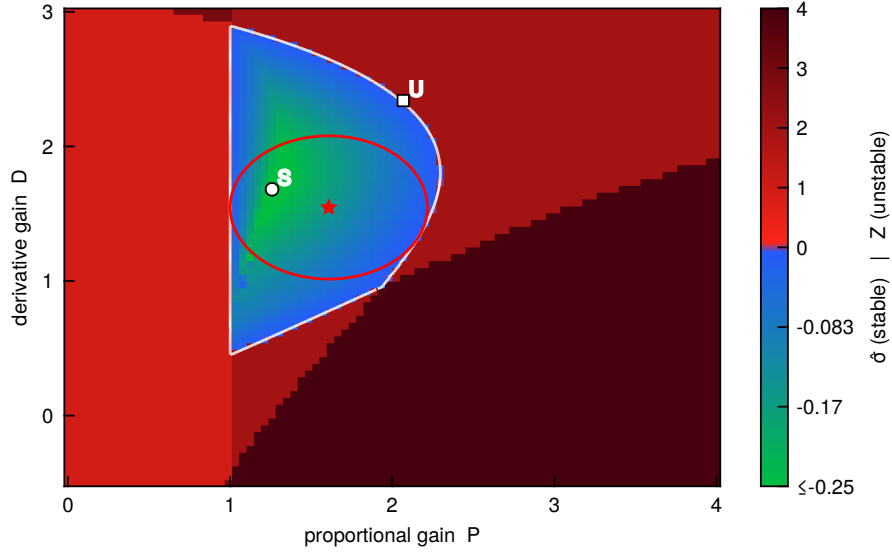


Figure 1: Stability chart of the constrained two-degree-of-freedom system with delayed PD control. Colored background: combined field (12) of the brute-force sweep (smooth spectral gap $\hat{\sigma}$ inside the stable domain, integer number of unstable roots outside). Black curve: stability boundary traced by MDBM with the scalar objective (13). Red ellipse and star: largest inscribed ellipse and the most robust parameter point. The markers S and U denote the stable and unstable points analyzed in Fig. 2.

(k_2, c_2), and an instrumentation block of mass m_3 is *rigidly bolted* to the absorber mass. Instead of eliminating the rigid connection by hand, the model keeps it as an algebraic constraint with the internal force F_c – exactly the form in which a simulation model of a larger assembly would be written. A digital controller measures the motion of the main mass and applies the delayed PD force $u(t) = -P x_1(t-\tau) - D \dot{x}_1(t-\tau)$ to it. With the state vector $\mathbf{z} = (x_1, x_2, x_3, v_1, v_2, v_3, F_c)$, the descriptor form (1) reads

$$\underbrace{\text{diag}(1, 1, 1, m_1, m_2, m_3, 0)}_{\mathbf{E}} \dot{\mathbf{z}}(t) = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ -k_1 x_1 - c_1 v_1 + k_2(x_2 - x_1) + c_2(v_2 - v_1) + u(t) \\ -k_2(x_2 - x_1) - c_2(v_2 - v_1) + F_c \\ -F_c \\ x_3 - x_2 \end{pmatrix}, \quad (16)$$

with the numerical values $m_1 = 1$, $m_2 = 0.3$, $m_3 = 0.2$, $k_1 = -1$, $k_2 = 1$, $c_1 = c_2 = 0.05$ and $\tau = 0.5$. The last row of (16) carries no time derivative – the mass matrix is singular – and the model is passed to the extraction of Section 5 exactly as written, in its simulation (right-hand-side) form. For verification, the constraint is also eliminated by hand

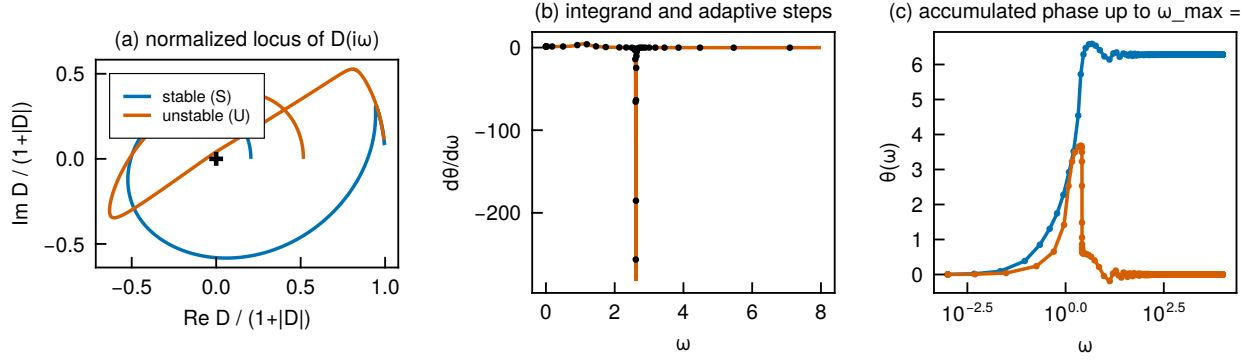


Figure 2: The method at the stable point S and unstable point U of Fig. 1. (a) Normalized locus of $\mathcal{D}(i\omega)$: the stable curve avoids the origin, the unstable one encircles it. (b) Phase slope $\theta'(\omega)$ with the accepted steps of the adaptive solver (dots): steps cluster automatically at the near-singular peaks that indicate close characteristic roots. (c) Accumulated phase up to $\omega_{\max} = 10^4$ on a logarithmic frequency axis: past the resonances the solver keeps resolving the slowly decaying (ω^{-1}) delay ripple of this velocity-feedback system with moderate, growing steps.

($x_3 = x_2$, merged mass $m_{23} = m_2 + m_3$), which yields the reduced quasi-polynomial

$$\mathcal{D}_{\text{red}}(\lambda) = \det \begin{pmatrix} m_1 \lambda^2 + (c_1 + c_2) \lambda + k_1 + k_2 + (P + D \lambda) e^{-\lambda \tau} & -(c_2 \lambda + k_2) \\ -(c_2 \lambda + k_2) & m_{23} \lambda^2 + c_2 \lambda + k_2 \end{pmatrix}. \quad (17)$$

The ratio of the automatically extracted 7×7 determinant to $\mathcal{D}_{\text{red}}$ is constant in λ to machine precision (deviation below 10^{-15} in the computed charts), which validates the mass-matrix extraction path end to end; the estimated leading order (7) is $n \approx 4.000$, as expected for two mechanical degrees of freedom. All results below were computed with the automatically extracted characteristic function.

6.2. The chart and the phase integration at work

Figure 1 shows the resulting stability chart over the control-gain plane (P, D) . Since the uncontrolled structure is unstable, the stable domain is a bounded island: its left boundary is the static (divergence) limit, while the remaining boundaries are Hopf-type curves whose crossing frequencies are delivered by $\hat{\omega}$. The combined coloring (12) displays the interior spectral gap – deep blue marks the fastest-decaying configurations – and the integer plateaus $Z = 1, 2, 3, 4$ outside, all in one map. The black MDBM curve is the high-resolution stability boundary traced from the scalar objective (13), and the red star marks the most robust gain combination found by the largest inscribed ellipse.

Figure 2 dissects the single-point computation at the two marked points. At the stable point S the locus of $\mathcal{D}(i\omega)$ avoids the origin and the accumulated phase settles at $\Delta\theta = n\pi/2$, giving $\tilde{Z} \approx 0$; at the unstable point U the locus encircles the origin and $\tilde{Z} \approx 2$. Panel (b) makes the central numerical argument of this paper visible: the accepted steps of the adaptive solver concentrate at the sharp peaks of θ' near the lightly damped resonances – precisely the features a fixed frequency grid tends to miss. Panel (c) shows the same march continuing over more than two further decades of frequency: since the delayed *velocity* feedback of this system produces a phase ripple decaying only like ω^{-1} (Section 3.1), the solver keeps taking moderate steps through the tail. This section is about accuracy, so

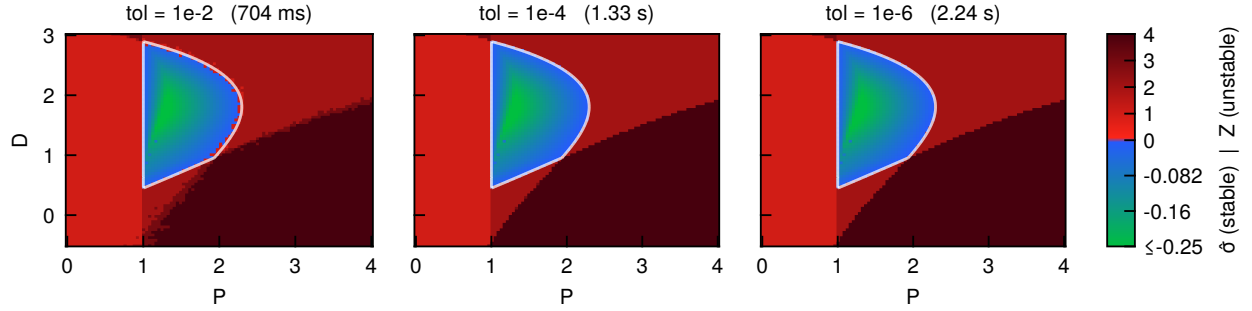


Figure 3: The showcase chart computed with solver tolerances 10^{-2} , 10^{-4} , 10^{-6} and 10^{-8} (left to right); the wall-clock time of each full sweep is given in the panel titles. The charts are visually near-identical even at 10^{-2} , where the count is already uncertain: the tolerance buys certainty rather than appearance, cf. Fig. 4 and Table 2. (The count Z is never refined – the Newton polish of Section 3.3 only touches the root estimate $\hat{\sigma}$ that colours the stable interior – so the residuals and miscount columns measure the integrator alone.) To isolate the integrator, this study uses $\omega_{\max} = 10^6$ (rather than the 10^4 that suffices for the chart) so that the truncated tail does not mask the two tightest tolerances, and evaluates the reduced form (17) of the same characteristic function, which is identical to the extracted one to machine precision but avoids the 7×7 determinant; Table 1 gives the cost of the extracted form.

Table 2: Tolerance study on the showcase system (grid as in Fig. 3): sweep time, integer residual statistics, the number of grid points where the rounded count would be ambiguous ($\varepsilon \geq 1/4$), the number whose Z actually deviates from the 10^{-8} -tolerance reference, and the subset of those that flip the stable/unstable classification. The “uncertain” and “wrong” columns measure genuinely different things: a skipped peak shifts $\tilde{Z}$ by exactly one and leaves it just as close to an integer, so a point can be wrong while looking certain – which is precisely why the cross-check (11) is needed.

tolerance	CPU time	mean ε	max ε	mean/tol	max/tol	uncertain	wrong Z	flips
10^{-2}	704.0 ms	$2.3 \cdot 10^{-2}$	$5.0 \cdot 10^{-1}$	2	50	151	290	57
10^{-4}	1.33 s	$1.8 \cdot 10^{-4}$	$5.4 \cdot 10^{-3}$	2	54	0	4	0
10^{-6}	2.24 s	$1.3 \cdot 10^{-5}$	$1.1 \cdot 10^{-4}$	13	115	0	0	0

the accuracy-critical studies below use the safe $\omega_{\max} = 10^6$; $\omega_{\max} = 10^4$ – three decades above the resonances – leaves a truncated tail of $\approx 0.2/\omega_{\max} \approx 2 \cdot 10^{-5}$ in $\tilde{Z}$, far below the rounding threshold, and is the economical choice for the charts themselves and for the gallery of Appendix Appendix A. Either way the truncation is exposed by the integer-residual diagnostic below, never hidden.

525 6.3. Tolerance as a universal accuracy dial

Figure 3 shows the identical chart computed with tolerances 10^{-2} , 10^{-4} , 10^{-6} and 10^{-8} , together with the measured sweep times, and Fig. 4 shows the corresponding integer-residual fields (10). The ladder deliberately starts *beyond* the safe range: at 10^{-2} the rule of thumb (18) predicts residuals of order one, i.e. genuine miscounts. Three observations matter for practice. First, the tolerance buys *certainty*, not appearance. The panels are visually near-identical, yet the loosest settings do miscount: measured against the 10^{-8} reference, the number of deviating grid points falls steeply with the tolerance (Table 2, last two columns), and this is worth spelling out because it is the one error the diagnostic

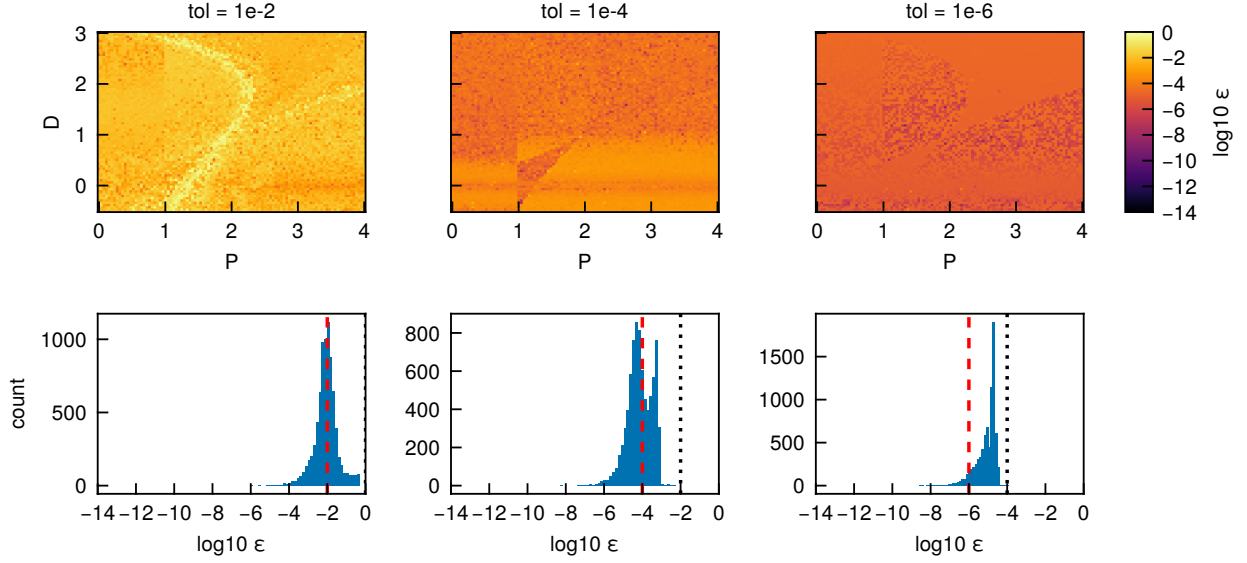


Figure 4: Self-validating error: fields and histograms of the integer residual $\varepsilon = |\tilde{Z} - \text{round}(\tilde{Z})|$, cf. (10), for the four tolerances of Fig. 3. Dashed red line: the requested tolerance; dotted black line: the practical rule $\varepsilon \approx 100 \text{ tol}$. The residual scales with the tolerance uniformly over the parameter plane, amplified by the accepted-step count of the frequency march.

of Section 3.4 cannot report. Every deviation is exactly ± 1 , and the corresponding integer residual stays far below the rounding threshold $1/2$ – the affected points look perfectly converged. They are skipped peaks, and they are the practical face of the caveat that Section 3.4 states in principle: at the loosest setting the residual field of Fig. 4 is uniformly healthy while a handful of pixels are nevertheless wrong. What does report them is the cross-check (11):

of the miscounts that flip the stable/unstable classification – the only ones a stability chart displays – it catches all, at the price of a few flagged pixels on a converged chart. Second – and this is the most useful outcome for a user – the integer residual tracks the requested tolerance across the whole parameter plane with a roughly constant factor, so that

$$\varepsilon \approx 100 \cdot \text{tol} \quad (18)$$

holds to within an order of magnitude over six decades (Table 2, last two columns). The factor is not universal – it is inherited from the number of accepted steps, and a march that resolves a slowly decaying ripple accumulates more of them – but it is remarkably stable across the chart and across the systems studied here. Equation (18) therefore lets the accuracy of Z be *prescribed* rather than discovered: a user who wants the count certain to 10^{-3} asks for 10^{-5} , which is exactly the package default. Third, the CPU-time premium of tightening the tolerance by six decades is well below two orders of magnitude, because the extra effort is spent only near the integrand peaks – and this asymmetry has a practical corollary worth stating plainly, since it runs against the instinct that a loose tolerance is dangerous.

A miscount is not the same kind of error as a displaced boundary, and for chart work it is the milder one. The two outputs of the sweep degrade completely differently with the tolerance: the count Z is a *rounded* quantity, so it fails discontinuously – an accumulated phase error of $\pm\pi$ flips it by one – whereas the tracked $\hat{\sigma}$ is a *local* quantity read off

the minimum of $|\mathcal{D}|$, and it inherits the tolerance smoothly. The traced stability boundary is built from $\hat{\sigma}$ alone through the objective (13), so it stays accurate even where the count does not: Table 2 lists the counts that flip, while the traced boundary of the same chart stays a small fraction of one grid pixel away from the reference trace at every tolerance studied (`data/boundary_shift.csv`). The practical recipe follows: *a loose tolerance is an appropriate tool for an initial search* – locating the stable island, its shape, and the robust operating point – after which the interesting region is recomputed at the default or tighter setting to certify the counts. What must not be done is to report a count from a loose sweep without the cross-check (11), because that is the one number whose error the residual will not reveal.

A flagged pixel need not be recomputed with a different backend: the march can repair itself (`peak_repair=true` in the package). The skipped peak is easy to *locate* even when it is far too narrow to *integrate*: its center is a minimum of $|\mathcal{D}(\sigma + i\omega)|^2$, and $|\mathcal{D}|^2$ dips toward that minimum over an $O(1)$ frequency range no matter how narrow the peak itself is. A root-finding callback therefore stops the integrator at every such minimum; the phase increment of the step that crossed it is recomputed by quadrature whose subdivision is forced into the located peak by a geometric ladder of breakpoints (an adaptive quadrature left to its own error estimate is blind to a peak narrower than the spacing of its first rule – the very blindness that skips peaks in the first place); and the march restarts exactly at the minimum, so that step control resolves the right half of the peak on its own. Measured across the Hopf point of this chart at default settings (`data/peak_repair.csv`): the plain march loses one count from $\Delta P = 10^{-6}$ inward on both sides of the boundary, and the Gauss–Kronrod backend – which must *discover* the peak – loses it silently at $\Delta P = 10^{-10}$, whereas the repaired march, which *knows* where the peak is, still counts correctly there. The price is $\approx 2.2\times$ the plain march (comparable to the quadrature backend at $2.2\times$), dominated by the per-step callback checks rather than by the repair quadrature itself, which is why the repair is off by default and the economical pattern is the one already implied: sweep plainly, flag with (11), and re-evaluate only the flagged pixels with the repair armed. The mechanism has a floor of its own: within about 10^3 floating-point ulps of the boundary the condition crossing can no longer be stepped past in double precision, and the callback disarms itself for the remainder of that march – a parameter point that close to the boundary is numerically *on* it.

Two limits deserve mention, since both are visible in Fig. 4 if the settings are pushed. The rule (18) holds only while the quadrature is the dominant error: below it lie the truncated tail (of order $0.2/\omega_{\max}$ for this velocity-feedback system, which is why $\omega_{\max} = 10^6$ is used here rather than the 10^4 that suffices for the chart itself) and the residual of the estimated leading order (7), of order 10^{-9} . Whichever of the three is largest sets the observed ε – and because all three are visible in the same number, the diagnostic tells the user which dial to turn next.

6.4. Rightmost roots and their refinement

Figure 5 quantifies the rightmost-root estimates. The raw tracked value $(\hat{\sigma}, \hat{\omega})$ – one Newton step evaluated at the integrand minimum – is near-exact on the stability boundary, accurate to a few percent in its vicinity, and degrades farther away, exactly as predicted in Section 3.2. What the timings settle is that the choice of refinement is essentially free: measured per parameter point against the complete solve, every variant adds only a few percent, because the

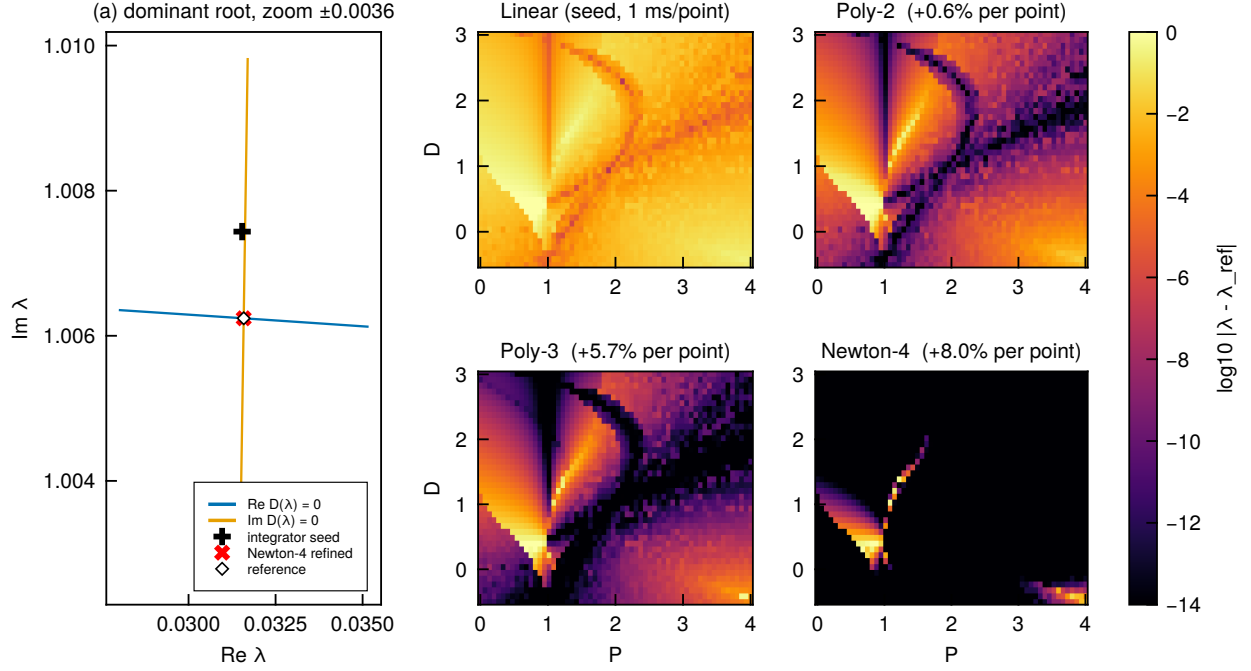


Figure 5: Rightmost-root accuracy. (a) Zoom on the dominant root at one parameter point. The two curves are the zero-level sets $\text{Re } \mathcal{D}(\lambda) = 0$ (blue) and $\text{Im } \mathcal{D}(\lambda) = 0$ (orange) in the complex λ -plane: a characteristic root satisfies both conditions simultaneously, i.e. it is exactly their intersection. Cross: raw tracked estimate $(\hat{\sigma}, \hat{\omega})$; red dot: the refined root; diamond: reference. (b–e) Error maps $\log_{10} |\lambda - \lambda_{\text{ref}}|$ of the refinement variants over the chart plane. Titles quote the cost *per parameter point of the complete solve*: the “Linear” panel gives the phase integration alone, the others the percentage this refinement adds on top of it.

refinement costs a handful of extra evaluations of $\mathcal{D}$ against the thousands the frequency march already spent. Four Newton iterations are in fact *cheaper* than a cubic Taylor solve – the latter needs finite differences and a companion-matrix eigenvalue problem – and are simultaneously the most accurate, reaching machine precision over almost the whole plane (Table 3). Since it is both better and cheaper, Newton-4 is the package default; the Taylor variants remain available and are the more conservative choice when a seed is poor, because they select the nearest root of a local polynomial rather than following the Newton flow. For stability charts the hierarchy is in any case exactly right: on the boundary, where the root position defines the chart, the estimates are near-exact; deep inside the domains, where only the sign matters, even the raw estimate suffices. Tracking $N_r > 1$ minima returns several dominant roots at once, which is what the multi-root recipe of Section 7.3 exploits.

6.5. Decay-rate design: σ -contours

Finally, Fig. 6 demonstrates the γ -stability capability: the traced σ -contours (Section 4.4) enclose progressively smaller regions with guaranteed spectral gaps, and they are consistent with the interpolated $\hat{\sigma}$ background computed from the coarse sweep alone. For control design this map answers directly which gain combinations guarantee a prescribed settling rate – at the cost of one extra boundary tracing per contour.

Table 3: Accuracy and cost of the root-refinement variants over the chart-plane sweep of Fig. 5 (reference: 15 Newton iterations). Times are per parameter point for the complete solve (integration plus refinement); the overhead column is measured against the unrefined solve.

refinement	median error	max error	time / point	overhead
Linear	$6.3 \cdot 10^{-3}$	$1.4 \cdot 10^2$	1.3 ms	–
Poly-2	$1.4 \cdot 10^{-7}$	$1.0 \cdot 10^2$	1.3 ms	+0.6%
Poly-3	$1.8 \cdot 10^{-10}$	$7.0 \cdot 10^1$	1.4 ms	+5.7%
Newton-4	$3.5 \cdot 10^{-17}$	$4.4 \cdot 10^1$	1.4 ms	+8.0%

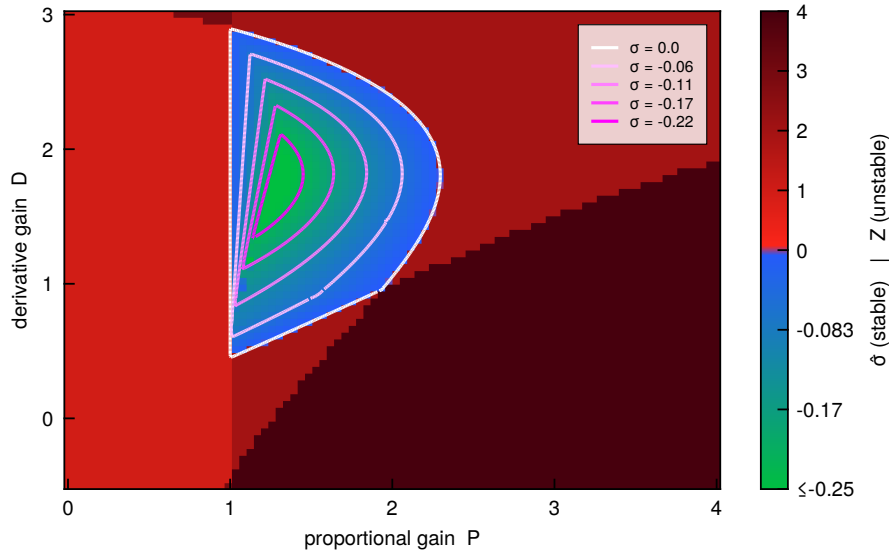


Figure 6: Decay-rate map of the showcase system: colored background – estimated spectral gap $\hat{\sigma}$ from the coarse sweep (stable domain only); curves – guaranteed γ -stability boundaries traced by MDBM for a family of shifted lines $\text{Re } \lambda = \sigma$. The two independent computations delineate the same landscape.

7. Performance analysis

7.1. Convergence and efficiency of the integrators

Figure 7 and Table 4 quantify the choice of back-end, and three points deserve emphasis. First, all adaptive back-ends track the requested tolerance over the studied range, with the count error settling at roughly two orders of magnitude above it; as long as that error stays below $1/2$ it coincides with the integer residual of Section 3.4, so this is the same rule of thumb $\varepsilon \approx 100 \text{ tol}$ quoted there. Second, the order of the pair matters far more than its family. The classical fifth-order pair is perfectly adequate at the default 10^{-5} , but at 10^{-3} it under-resolves the ω^{-1} ripple of this velocity-feedback system so badly that $\tilde{Z}$ is off by 0.8 – within a hair of losing the count – whereas the ninth-order default [64] is three orders of magnitude more accurate there and, at tolerances below 10^{-6} , also cheaper. The linearly implicit Rosenbrock pair [72, 73, 74] is uniformly dominated, as expected once one recognizes that the problem has

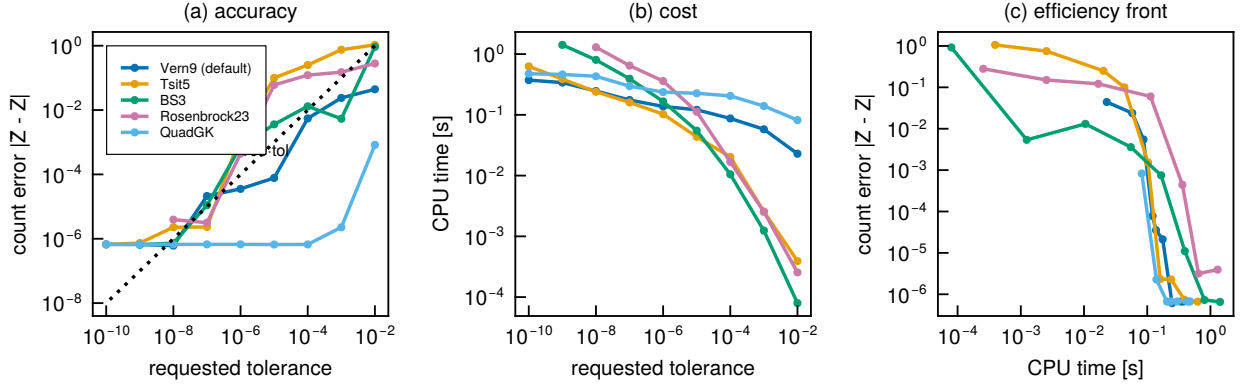


Figure 7: Integration back-ends at an unstable point of the showcase system, where the exact count $Z = 2$ is known (verified independently by a high-resolution semi-discretization eigenvalue count), so the plotted count error $|\tilde{Z} - Z|$ is an *absolute* error measure requiring no reference. (a) Accuracy versus requested tolerance; the dotted line marks the practical rule error $\approx 100 \text{ tol}$. (b) Cost versus tolerance. (c) Efficiency front. Error is on the vertical axis in both (a) and (c). A curve ends where a single parameter point exceeded a cost cap of 20 s, which is itself informative.

Table 4: Count error $|\tilde{Z} - Z|$ (left block) and single-point CPU time (right block) of the back-ends at selected tolerances; system and point as in Fig. 7.

tol	Vern9 (default)	Tsit5	BS3	Rosenbrock23	QuadGK	Vern9 (default)	Tsit5	BS3	Rosenbrock23
10^{-2}	—	—	—	—	—	—	—	—	—
10^{-4}	$5.5 \cdot 10^{-3}$	$2.5 \cdot 10^{-1}$	$1.3 \cdot 10^{-2}$	$1.2 \cdot 10^{-1}$	$6.7 \cdot 10^{-7}$	87.2 ms	20.3 ms	10.5 ms	16.8 ms
10^{-6}	$3.6 \cdot 10^{-5}$	$1.5 \cdot 10^{-3}$	$7.5 \cdot 10^{-4}$	$4.4 \cdot 10^{-4}$	$6.7 \cdot 10^{-7}$	137.5 ms	102.8 ms	165.9 ms	359.8 ms
10^{-8}	$6.1 \cdot 10^{-7}$	$2.3 \cdot 10^{-6}$	$7.3 \cdot 10^{-7}$	$3.9 \cdot 10^{-6}$	$6.7 \cdot 10^{-7}$	246.8 ms	239.6 ms	803.2 ms	1.30 s

no stiffness for it to exploit (Section 3.1); at 10^{-9} it needs tens of seconds for a single parameter point where the ninth-order pair needs a fraction of one, which is why the sweep in Fig. 7 stops each method once a point exceeds a cost cap. Third, at the practically relevant setting 10^{-5} the back-ends are within a small factor of one another in both accuracy and cost: *the choice of integrator is simply not critical there*, and the default is chosen for graceful degradation rather than for speed.

The one situation in which the back-ends genuinely differ is the immediate vicinity of a stability boundary, and the outcome is the opposite of what the folklore about adaptive solvers suggests (Table 5). As the crossing root approaches the integration line, the integrand peak narrows in proportion to $|\hat{\sigma}|$; at $\Delta P = 10^{-8}$ it is $6.6 \cdot 10^{-10}$ wide and $1.5 \cdot 10^9$ tall, while 10^{-3} away from it the integrand is back to its smooth background. The marching solver then steps clean over the peak, loses exactly $\pm\pi$, reports one root too few – and returns a success flag and a perfectly healthy integer residual. Gauss–Kronrod survives the same test, though not by cleverness: the persistent delay ripple forces it to subdivide finely across the whole range anyway, and that global refinement happens to resolve the spike. Neither method carries a guarantee, since both sample pointwise – and pressed one step further, both indeed fail: at

Table 5: Approaching a Hopf boundary of the showcase system at distance ΔP in the gain. The peak half-width equals $|\hat{\sigma}|$; the marching solver silently loses one root once the peak falls below its step resolution, while the integer residual stays small ($\bar{Z}$ shifts by exactly one).

ΔP	$\hat{\sigma}$ (peak half-width)	Z march	Z quadrature	ε march	ε quadrature
$1 \cdot 10^{-2}$	$6.6 \cdot 10^{-4}$	4	4	$1.6 \cdot 10^{-5}$	$4.1 \cdot 10^{-7}$
$1 \cdot 10^{-4}$	$6.6 \cdot 10^{-6}$	4	4	$1.6 \cdot 10^{-5}$	$4.1 \cdot 10^{-7}$
$1 \cdot 10^{-6}$	$6.6 \cdot 10^{-8}$	4	4	$1.3 \cdot 10^{-5}$	$4.1 \cdot 10^{-7}$
$1 \cdot 10^{-8}$	$6.6 \cdot 10^{-10}$	3	4	$9.1 \cdot 10^{-6}$	$4.1 \cdot 10^{-7}$
$1 \cdot 10^{-10}$	$6.6 \cdot 10^{-12}$	3	4	$8.3 \cdot 10^{-6}$	$5.2 \cdot 10^{-7}$

620 $\Delta P = 10^{-10}$ the peak escapes the ripple-driven refinement as well, and Gauss–Kronrod returns a clean residual with the wrong count (`data/peak_repair.csv`), while the repaired march of Section 6.3 – which does not *search* for the peak but is steered onto it by the $|\mathcal{D}|^2$ minimum – still counts correctly there.

The practical consequences are, however, mild, and worth spelling out so the phenomenon is neither ignored nor overstated. First, it requires landing within $\sim 10^{-8}$ of the boundary in the gain – a measure-zero event that a chart
625 grid hits, if ever, in a single pixel, and the damage there is a count off by one in a field that varies by one anyway, not a wild outlier. Second, the MDBM trace of Section 4.2 is insensitive to it in a quantifiable way, even though it deliberately converges onto the boundary: its objective $\text{sign}(Z=0) |\hat{\sigma} - \sigma|$ tends to zero at the boundary *regardless* of the sign, because $|\hat{\sigma} - \sigma|$ does, so a miscounted Z can at worst flip the sign inside the strip where the peak is skipped – displacing the interpolated boundary point by no more than that strip’s width, here $\sim 10^{-8}$ in the gain, far below
630 the plotting resolution of any chart. Third, where the count itself matters the tracked root settles the question: at $\Delta P = 10^{-8}$ it still reports $\hat{\sigma} = +6.6 \cdot 10^{-10}$, correctly placing a root to the right of the axis, so the neighbourhood-restricted sign test (11) exposes the inconsistency for free – and this is not a hypothetical safeguard: on the tolerance study of Section 6.3 it catches every miscount that flips the stable/unstable classification. Since that test is available only from the ordered march, the march remains the default; quadrature is the better choice when only Z is wanted
635 and the characteristic function has a rapidly decaying tail (Section 7.4).

7.2. Chart-level cost, memory and cold start

Two operating points matter in practice, and Table 7 separates them because conflating them is what makes performance claims about stability charts unfalsifiable. With the *safe* settings – $\omega_{\max} = 10^6$ and $\text{tol} = 10^{-5}$ – a complete
640 100×100 chart of a scalar characteristic function costs a fraction of a second, and every one of the ten thousand counts matches a tight reference. With an *exploratory* preset – $\omega_{\max} = 10^4$, $\text{tol} = 10^{-4}$ – the same grid resolves about twice as fast, and on those systems still without a single wrong count.

Two settings decide the cost of a chart, and neither is a property of the counting method – both are choices the user makes. They are worth quantifying on the showcase DDAE, because together they span three orders of magnitude.

Table 6: Warm chart-level costs (median of repeated runs, 16 threads): full brute-force sweeps for three systems of increasing evaluation cost, with allocated memory. The last column counts the points at which the back-end’s Z deviates from a tight-tolerance reference of the same chart: the adaptive back-ends are required to match everywhere before their time is recorded, while the fixed-step back-end – whose resolution is set by hand – reports its misses honestly.

system	back-end	chart (100×100)	per point [μ s]	alloc [MB]	wrong Z
2nd-order, no delay	ODE Vern9 + root (default)	54.7 ms	5	40	0
2nd-order, no delay	ODE Vern9, Z only	45.5 ms	5	17	0
2nd-order, no delay	QuadGK, Z only	33.4 ms	3	27	0
2nd-order, no delay	fixed-step (500)	38.3 ms	4	1	4324
4th-order + delay	ODE Vern9 + root (default)	410.2 ms	41	40	0
4th-order + delay	ODE Vern9, Z only	401.8 ms	40	17	0
4th-order + delay	QuadGK, Z only	181.1 ms	18	38	0
4th-order + delay	fixed-step (500)	103.3 ms	10	1	4556
2-DOF DAE showcase	ODE Vern9 + root (default)	1.93 s	193	40	0
2-DOF DAE showcase	ODE Vern9, Z only	1.85 s	185	17	0
2-DOF DAE showcase	QuadGK, Z only	2.64 s	264	407	0
2-DOF DAE showcase	fixed-step (500)	84.4 ms	8	1	6097

The first is $\omega_{\max}$: this system’s delayed *velocity* feedback makes the phase ripple decay only like ω^{-1} , so every extra decade must genuinely be resolved, and $\omega_{\max} = 10^6$ costs some sixty times $\omega_{\max} = 10^4$ – while buying nothing, since the tail truncated at 10^4 contributes $\approx 0.2/\omega_{\max} = 2 \cdot 10^{-5}$ to $\tilde{Z}$, five orders of magnitude below the rounding threshold. The second is the form of $\mathcal{D}$: the automatic extraction is a factor of 20 dearer per evaluation than the hand-derived expression (Table 1). Choosing badly on both counts turns the 1.89 s chart of Table 7 into a fifty-minute one. This is what the asymmetric guidance of Section 3.1 means in practice – be generous with $\omega_{\max}$ when the ripple decays fast, where it is nearly free, and frugal when it does not.

Table 6 reports the cost of a *complete* 100×100 chart – ten thousand parameter points – on a AMD Ryzen 7 5800H with Radeon Graphics (16 threads, 15 GB RAM, Julia 1.12.4, package revision f194d23), with the per-point cost listed alongside so that other resolutions can be scaled directly. For scalar characteristic functions a full chart takes tens to hundreds of milliseconds, and the entire hybrid workflow (colored sweep plus MDBM boundary) finishes well inside one second – genuinely interactive. The last column is worth a glance: the fixed-step back-end is the fastest entry for the delayed systems and gets thousands of the ten thousand counts wrong, for the reason given in Section 3.5. The showcase DDAE is listed with its hand-derived characteristic function, as are all figures in this paper; what the automatic extraction of Section 5 would cost instead is measured separately in Table 1. Memory stays modest throughout because the phase ODE carries a single scalar state and the sweep stores only the five result fields; no large

Table 7: The two operating points of the method on the same 100×100 chart: the *accurate* preset (the safe defaults) and the *fast* preset used for the exploratory charts of Appendix Appendix A. The last column counts the points whose Z differs from a tight reference, i.e. the price of the fast preset, which buys roughly an order of magnitude in time.

system	preset	$\omega_{\max}$	tol	chart (100×100)	per point [μ s]	wrong Z
2nd-order, no delay	accurate	10^4	10^{-5}	55.4 ms	6	0
	fast	10^4	10^{-3}	43.8 ms	4	0
4th-order + delay	accurate	10^4	10^{-5}	419.6 ms	42	0
	fast	10^4	10^{-3}	213.9 ms	21	14
2-DOF DAE showcase	accurate	10^4	10^{-5}	1.93 s	193	0
	fast	10^4	10^{-3}	1.12 s	112	32

Table 8: What the MDBM boundary trace buys: from an initial mesh with the listed refinement levels, the traced boundary has the equivalent resolution of a uniform grid of the given size, at the listed number of function evaluations. The saving factor is the ratio of the two point counts.

system	initial mesh	levels	equiv. resolution	evaluations	boundary pts	time	saving
2nd-order, no delay	20×20	4	320×320	1988	411	46.6 ms	$52 \times$
4th-order + delay	20×20	4	320×320	3462	792	218.3 ms	$30 \times$
2-DOF DAE showcase	20×20	4	320×320	5613	1374	1.27 s	$18 \times$

matrices are formed at any point. Cold start is a one-time cost dominated by package loading: in a fresh Julia process, loading takes 7–12 s and the first thousand-point sweep 0.15–0.26 s thanks to the precompilation workload shipped with the package, after which every further chart runs at the warm speeds of Table 6.

Table 8 makes the boundary-tracing argument of Section 4.2 concrete. Four refinement levels from a 20×20 mesh yield a boundary resolved as finely as a 320×320 uniform grid would resolve it, while evaluating only the cells that actually bracket the boundary – a saving of 20 to 50 times in function evaluations, which is precisely why the high-resolution boundary is affordable on top of a deliberately coarse colored background. These numbers substantiate the practical claim of the paper: the stability chart of a nontrivial delayed system refreshes fast enough to be explored *interactively*, which changes how such charts are used in design – parameter studies and optimization loops can call the entire chart pipeline as an inner function.

7.3. Comparison with semi-discretization

Semi-discretization [9] – here in its efficient library implementation with the multiplication-free variant [25, 26] – approximates the dominant characteristic exponent through the spectral radius of a finite transition matrix, converging polynomially in the number of discretization steps per delay. Table 9 compares it with the present method on the reduced showcase system at a stable, a near-boundary and an unstable point. The dominant root of the present

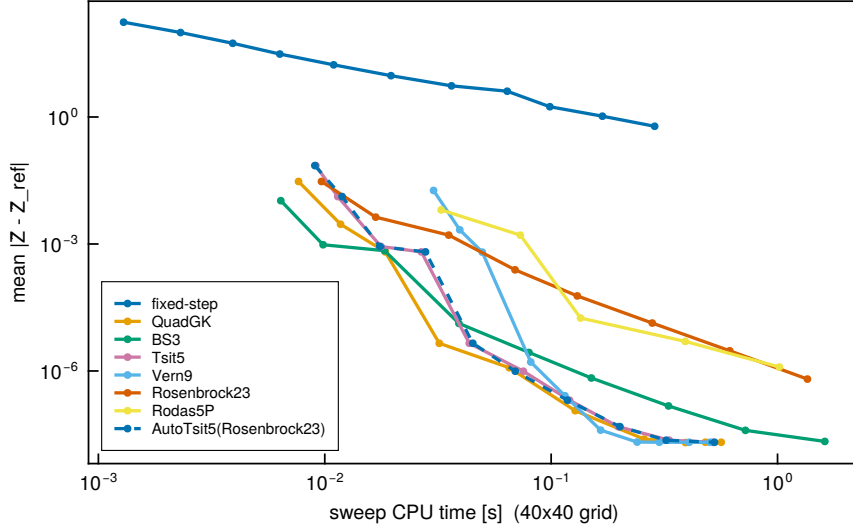


Figure 8: Error–cost Pareto fronts of eight integration back-ends on a 40×40 sweep of the fourth-order benchmark oscillator (Appendix Appendix A): fixed-step trapezoid (step-count sweep), adaptive Gauss–Kronrod, and six ODE solvers (tolerance sweeps).

method is obtained by the multi-root recipe of Section 3.2: ten tracked minima, all refined, maximal real part. That the discretization eigenvalues and the tracked quasi-polynomial root converge towards the same residual-certified reference is the most valuable outcome of the comparison – it validates both formulations against each other.

Beyond that agreement the two methods are not commensurable, and it would be misleading to plot them on shared error axes: their accuracy is governed by different dials. This is the substantive difference. Semi-discretization requires the user to choose the number of steps per delay *by hand*; the discretization is not adaptive, and an insufficient resolution returns a confidently wrong exponent with no warning and no internal indication that anything is amiss. The present method sets its own frequency steps from the behaviour of the system, exposes the accuracy of the root through the requested tolerance (Section 3.4), and ships two independent consistency checks: the integer residual, which certifies the quadrature, and the sign agreement between $\hat{\sigma}$ and Z , which certifies the count. A user who wants a more accurate root turns one dial and can verify the result; a user of the discretization approach must re-run at several resolutions and compare. The comparison is also intentionally asymmetric in what the methods deliver: semi-discretization returns the full dominant multiplier cluster and extends unchanged to time-periodic systems, where the argument-principle formulation does not apply (Section 8); the present method returns exactly what an *autonomous* stability chart needs – the root count, the boundary, and the dominant root on demand – at interactive speed.

7.4. A broader solver comparison

Table 10 isolates what actually drives the cost, and the numbers are more sobering than the folklore on either side. The decisive quantity is not the back-end but the decay rate of the phase ripple, which is set by how far the delayed terms sit below the leading power of $\mathcal{D}$. Where the ripple decays fast – delayed *position* feedback, $\theta' \sim \omega^{-3}$ –

quadrature recognizes the negligible tail panel by panel and finishes in roughly 570 evaluations even for $\omega_{\max} = 10^6$, about 2.5 times fewer than the march spends walking there. Where the ripple decays slowly – delayed *velocity* feedback, $\theta' \sim \omega^{-1}$ – the ranking reverses: both back-ends must resolve the whole range, and the march is then some three times *cheaper* than the quadrature. So neither back-end dominates; they stay within a factor of about three of each other in both directions, and the lesson for a user is to spend attention on $\omega_{\max}$ and on the structure of the feedback, not on the choice of integrator.

One implementation detail underpins those quadrature numbers and is worth stating, because it is a trap rather than a tuning knob: the Gauss–Kronrod back-end is seeded with logarithmically spaced interior breakpoints, one per two frequency decades. Without them the *first* G_7 – K_{15} panel spans the entire window $[0, \omega_{\max}]$, its lowest node lands at $\omega \approx 0.004 \omega_{\max}$ – above the whole resonance region for $\omega_{\max} = 10^6$ – and the smooth, fast-decaying tail then passes the error test after fifteen evaluations. The routine returns $\tilde{Z} = n/2$ having never seen a single root, with an integer residual of *exactly* zero: the self-diagnostic of Section 3.4 cannot flag it, because nothing about the quadrature failed. The breakpoints cost a few dozen extra evaluations and remove the failure mode entirely.

Figure 8 extends the comparison to a wider selection of ODE solvers on the fourth-order benchmark system, and one of its lessons is negative enough to be worth stating plainly. (i) The fixed-step trapezoid is *dominated everywhere* on this problem: matching the mean accuracy that the ninth-order pair reaches at its loosest setting already costs it more than an order of magnitude in time, and even at $3 \cdot 10^4$ steps its worst error over the chart remains several roots, because a uniform grid cannot resolve a peak that narrows without bound as a root approaches the line. Its zero-overhead appeal is real only where the integrand has no such peak. (ii) All adaptive methods follow their tolerance. (iii) Higher-order pairs dominate at tight tolerances, while the linearly implicit pair is uniformly dominated – there is no stiffness for it to exploit. The spread between the *adaptive* methods is far smaller than the gap that separates all of them from a fixed grid, and smaller still than the gap to methods that rebuild discretization matrices at every parameter point, which is the practical message of this section.

8. Conclusions and outlook

The Nyquist/argument-principle criterion for linear autonomous time-delay systems was reformulated as a scalar ordinary differential equation for the phase of the characteristic function and marched by a high-order adaptive integrator with an exact, automatically differentiated integrand. This reformulation tames the practically unbounded frequency range and the near-singular integrand through the mature step-size control of modern ODE software, and it makes the accuracy self-validating through the integer residual of the computed winding number, a dimensionless error measure that scales with the requested tolerance – empirically as $\varepsilon \approx 100 \text{ tol}$ – so that the accuracy of the count can be prescribed rather than discovered. It should be said plainly that the phase ODE carries no stiffness: its right-hand side is independent of the state, so the formulation is a quadrature in disguise, and adaptive quadrature is fully competitive for the count alone. What the ordered march uniquely provides – and what every chart in this paper is

built from – is the characteristic root obtained along the way. The location of the characteristic root closest to the integration line is obtained during the same sweep at negligible cost and is refined to high accuracy – essentially to machine precision near the stability boundary – by Newton or cubic Taylor corrections built on the same automatic derivatives.

The root estimate transforms the integer-valued root-counting map into an interpolable chart: one coloring simultaneously shows the number of unstable roots and the spectral gap of the stable domain; the estimated root distance serves as a scalar objective for the multi-dimensional bisection method, which traces the true stability boundary (rather than all root-crossing D-curves) at high equivalent resolution; the most robust parameter point follows from the largest inscribed circle; and shifted integration lines yield guaranteed decay-rate (γ -stability) contours. The characteristic function itself is extracted algorithmically from a simulation-ready right-hand side – including nonlinear models (linearized automatically) and singular mass matrices – so the complete workflow requires nothing beyond the model and the parameter ranges. The case studies demonstrate the identical code path on retarded, neutral, distributed-delay, differential–algebraic, transcendental, high-dimensional and fractional-order systems, at chart-generation speeds that allow interactive exploration and optimization.

One modelling lesson deserves to be carried away from the case studies, because it costs nothing and outweighs any tuning of the numerics: where a model admits a natural open-loop/feedback split, the characteristic function should be written as the return difference $1 + L(\lambda)$ rather than as a raw determinant. The zeros are the same, the added poles are the open-loop roots and lie in the left half-plane for an open-loop stable structure, and the leading order becomes exactly zero – which removes both the need to estimate it and the astronomical magnitudes that a high-order determinant produces.

The limitations delineate the outlook. First, the method is restricted to autonomous (time-invariant) linear systems: for time-periodic coefficients the characteristic function would have to be replaced by a Hill-type infinite determinant, whose robust truncation is an open issue, and time-domain discretization methods such as semi-discretization [9, 26] remain the more practical choice there. Second, away from the stability boundary the rightmost-root estimate is of first order unless refined, so the method complements rather than replaces full spectral solvers when the deep spectrum is needed. Third – and this is intrinsic to the argument principle rather than to any implementation of it – a peak that no sample lands on costs exactly $\pm\pi$ and shifts the count by one while leaving it an integer, so the residual cannot see it; sufficiently close to a boundary the marching solver does step over such peaks. The cross-check between the count and the independently tracked root detects this at no cost – provided it is restricted to a neighbourhood of the integration line, since farther away the tracked root need not be the dominant one – and the affected classification is confined to the crossing of the last unstable root; the optional detect-and-repair callback of Section 6.3 then mends the flagged pixels within the march. A formally guaranteed count would still require a different device – interval arithmetic, or an a-priori bound on the peak locations. Fourth, for neutral systems beyond the essential instability threshold the root count diverges with the truncation frequency; the classification into stable and unstable domains nevertheless remains correct, as demonstrated in Appendix Appendix A. Finally, the fractional-order results are preliminary in

the sense that the argument-principle counting on the principal branch was validated here numerically – including the reproduction of a published fractional-controller design chart – but a rigorous treatment of branch-cut effects, in particular for shifted integration lines, is left for future work. Finally, for large models the cost is dominated not by the method but by each evaluation of $\mathcal{D}$: the gallery’s scalar systems yield a 100×100 chart in a fraction of a second, whereas the finite element and dense-matrix cases spend their time inside a factorization per frequency. Where the feedback has low rank the matrix determinant lemma already removes most of that cost (A.3); caching or updating the factorization across frequencies, and exploiting sparsity, are the natural next steps. Extending the automatic extraction to neutral right-hand sides and coupling the interactive charts to formal optimization loops are further continuations.

Software and reproducibility

The method is implemented in the open-source Julia package `InterpolatedNyquist.jl` (MIT license, <https://github.com/bachrathyd/InterpolatedNyquist.jl>), built on `OrdinaryDiffEq.jl` [66], `ForwardDiff.jl` [67], `QuadGK.jl` [71], `MDBM.jl` [51] and `DelaunayTriangulation.jl` [75]. All figures and tables of this paper can be regenerated by the scripts in the `paper/` directory of the repository; the benchmark hardware and software versions are embedded automatically in the manuscript.

Funding

The author disclosed receipt of the following financial support for the research, authorship and publication of this article: this work was supported by the National Research, Development and Innovation Fund of Hungary under project no. 152125 (ENSURE: Efficient Numerical Solver for Universal Real-Time Evaluation of Stability and Periodic Motion).

Declaration of competing interest

The author declares that there is no known competing financial interest or personal relationship that could have appeared to influence the work reported in this paper.

Appendix A. Case-study gallery

This appendix demonstrates the identical workflow of Section 4.5 – coarse colored sweep, MDBM boundary, combined coloring (12) – on eleven further systems covering all supported model classes, plus the reproduction of a published design chart. The complete scripts, grid resolutions and timings are part of the package repository.

These charts are the speed argument.. Sections 6 and 7 make the accuracy case, with tolerances down to 10^{-8} . The gallery answers the complementary question – *what does a chart cost when one simply wants to see the picture?* – and so every panel here is computed at settings chosen for speed: a generous 100×100 background grid, and a tolerance of 10^{-4} , one decade looser than the package default. Each panel is annotated with the measured wall-clock cost of both stages. At these settings the count of a few boundary-adjacent pixels may be off by one (Section 6.3); that is precisely the trade a first exploration should make, it is invisible at chart resolution, and the traced boundary – which is built from $\hat{\sigma}$, not from the count – is unaffected. The MDBM traces start from a 7×7 mesh and refine until the boundary carries roughly four times the resolution of the background grid (448×448 -equivalent on the 100×100 panels), from 49 blanket evaluations.

$\omega_{\max} = 10^4$, the standard window of this paper, is used wherever it helps. Five panels use less, for reasons of substance: the phase ripple of a *neutral* system (A.4–A.6) does not decay at all, so a wider window buys no accuracy whatsoever – what bounds the error is $\arcsin |a|/\pi$, not $\omega_{\max}$ – while for the beam and the finite element bar (A.8–A.9) the Kelvin–Voigt damping makes the integrand vanish like $e^{-\sqrt{\omega/2c}}$ long before $\omega = 200$. Rather than assert that these choices are adequate, each panel reports the measured integer residual (10) of its own grid (data/gallery_timings.csv): on every *retarded* panel the median residual is $\sim 10^{-4}$ and at most a handful of the 10^4 points are uncertain, which is what a well-chosen $\omega_{\max}$ looks like. On the neutral panels the median is ≈ 0.1 – the irreducible tail oscillation discussed in Section 3.4, not a symptom of truncation.

Reading the charts.. Every panel uses the same convention, the combined field C of (12): $\hat{\sigma}$ where $Z = 0$, and Z elsewhere. Outside the stable domain the colour encodes the *integer* number of characteristic roots to the right of the imaginary axis, so those regions appear as flat plateaus with sharp steps between them. Inside the stable domain ($Z = 0$) the same colour axis continues *below zero* and encodes the estimated decay rate $\hat{\sigma} < 0$, i.e. the spectral gap: the darker the shade, the faster every disturbance decays, and the colour varies smoothly because $\hat{\sigma}$ does. One map therefore answers three questions at once – where the system is stable, how robustly, and how many roots must be stabilized elsewhere. Following Section 3.2, the shading uses the *dominant* root – five tracked minima, all refined, maximal real part – rather than the single closest minimum, which would switch branch where another mode overtakes it and shade the deep stable domain with a non-dominant root. The black curve is the stability boundary traced by MDBM from the objective (13), not a contour of the coloured grid; the same dominant-root convention makes that objective continuous across a branch crossing, which matters more than it sounds – a jump there lets MDBM’s linear in-cell interpolation invent a spurious boundary point in the middle of the stable domain. Each panel is annotated with the wall-clock cost of both stages: the brute-force sweep that produces the colour, and the MDBM trace that produces the boundary.

A.1 Fourth-order delayed oscillator.. $\mathcal{D}(\lambda) = c_1 \lambda^4 + \lambda^2 + 2\zeta \lambda + 1 + (P + D\lambda)e^{-\lambda\tau}$ with $c_1 = 0.03$, $\zeta = 0.02$, $\tau = 0.5$, swept over the control gains (P, D) . This is the regression benchmark of the package (test suite and performance history) and the system of the solver comparison in Fig. 8.

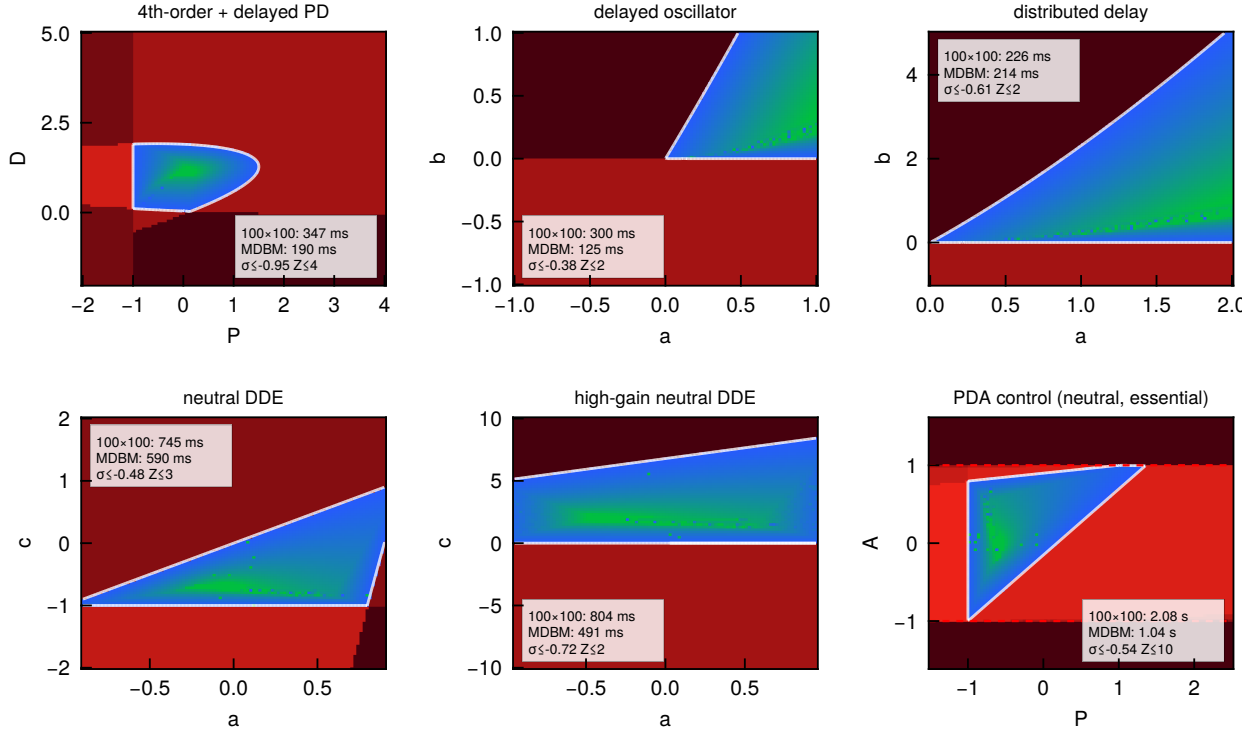


Figure A.9: Case studies, part I: (top row) fourth-order delayed oscillator, delayed oscillator, distributed delay; (bottom row) neutral DDE, high-gain neutral DDE, and PDA control with essential instability (the analytic essential boundaries $|A| = 1$ are dashed red).

A.2 Delayed oscillator. $\mathcal{D}(\lambda) = \lambda^2 + a\lambda + b e^{-\lambda/\tau}$ over (a, b) : the textbook retarded case with both divergence and Hopf boundaries.

A.3 Distributed delay. $\mathcal{D}(\lambda) = \lambda^2 + a\lambda + b(1 - e^{-\lambda\tau})/\lambda$, $\tau = 1$, i.e., a uniform delay kernel $\int_0^\tau x(t - \vartheta) d\vartheta$. The kernel integral appears in closed form in $\mathcal{D}$; the removable singularity at $\lambda = 0$ is handled by the series fallback in the model code and by the origin offset of Section 3.1.

A.4–A.5 Neutral systems. $\mathcal{D}(\lambda) = \lambda^2 + a\lambda^2 e^{-\lambda} + 1 + c e^{-\lambda}$ and its high-gain damped variant $\mathcal{D}(\lambda) = \lambda^2 + a\lambda^2 e^{-\lambda} + 5\lambda + c e^{-\lambda}$, swept over (a, c) with $|a| < 1$. The delayed acceleration term makes the equations neutral: the phase integrand oscillates persistently up to arbitrarily high frequencies, so the truncation is kept moderate ($\omega_{\max} = 200$) – the resulting tail error is bounded by $\arcsin |a|/\pi < 1/2$ and never corrupts the rounded count – and the leading-order fit (7) delivers the effective order $n = 2$ where the naive point estimator diverges.

A.6 PDA control and essential instability. Acceleration feedback is popular in active vibration control, but the delayed acceleration term $A\ddot{x}(t - \tau)$ turns the closed loop into a neutral system with characteristic function

$$\mathcal{D}(\lambda) = \lambda^2 + 2\zeta\lambda + 1 + (P + D\lambda + A\lambda^2) e^{-\lambda\tau},$$

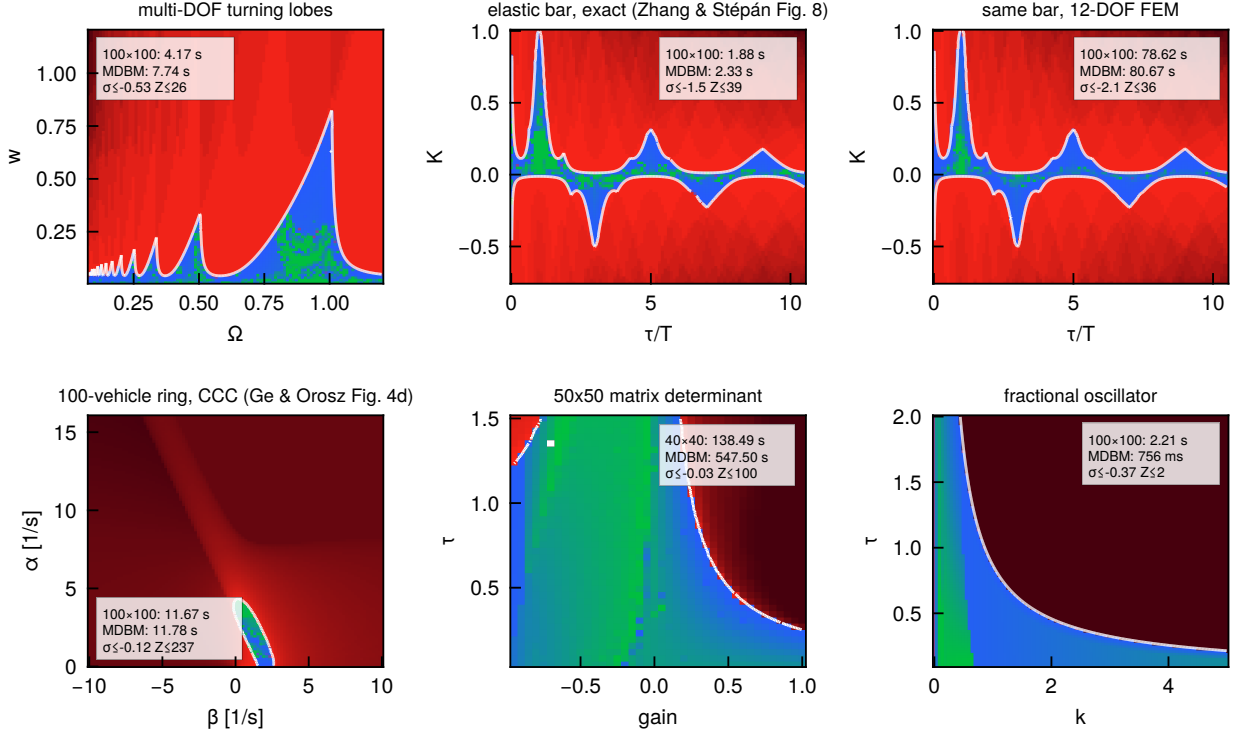


Figure A.10: Case studies, part II: (top row) multi-mode turning lobes, transcendental beam, 29-DOF finite element bar; (bottom row) 50 $\times$ 50 matrix determinant, fractional-order oscillator.

whose neutral coefficient is $1 + Ae^{-\lambda\tau}$. For $|A| > 1$ the essential spectrum moves to $\text{Re } \lambda = \ln |A|/\tau > 0$: infinitely many roots become unstable and $Z \rightarrow \infty$. With a finite $\omega_{\max}$ the computed count saturates at a large value ($\sim \omega_{\max} \tau/(2\pi)$), so the method cannot – and need not – report the exact count there; crucially, the *classification* into stable and unstable domains remains correct, and the chart in Fig. A.9 displays both the regular Hopf-type lobes and the analytic essential boundaries $|A| = 1$, which the MDBM trace locates without special treatment. The count is capped in the plot for readability. This case study substantiates the claim of Section 8 that essential loss of stability is detected robustly even though the count itself diverges.

A.7 Multi-mode turning lobes.. The regenerative turning model with a two-mode flexible structure, $\mathcal{D}(\lambda) = 1 + w(1 - e^{-2\pi\lambda/\Omega}) \sum_{k=1}^2 g_k/(\lambda^2 + 2\zeta_k\omega_k\lambda + \omega_k^2)$, over spindle speed Ω and chip width w [1, 3, 52]. Two features deserve emphasis. First, $\mathcal{D}$ is *rational* – its structural poles lie in the open left half-plane, so the argument-principle count is unaffected, and $\mathcal{D} \rightarrow 1$ at high frequency gives an effective order $n \approx 0$; no polynomial form is ever constructed. Second, the chart makes the introductory argument visible: towards low spindle speeds the lobes accumulate, and a D-curve-based method would trace an unbounded family of curves of which almost none bounds the stable region, whereas the boundary objective (13) follows only the true stability limit [54].

A.8 Transcendental beam, and why the damping model decides everything.. Longitudinal vibration of an elastic bar with delayed boundary feedback gives – *without any spatial discretization* – the exact transcendental return difference

$$\mathcal{D}(\lambda) = 1 + K_p e^{-\lambda\tau} \operatorname{sech}(\gamma(\lambda)L), \quad L = 1, \quad (\text{A.1})$$

cf. the exact analysis of Zhang and Stépán [69] and the finite-DOF hierarchy of Kidd and Stépán [76]. Whether a finite count even exists is decided entirely by the damping model, and this is the instructive point of the example. With external (viscous) damping, $\gamma = \sqrt{\lambda^2 + c\lambda}$, every mode is damped equally: the closed-loop roots accumulate along a vertical line $\operatorname{Re} \lambda \rightarrow \text{const}$, so the number of unstable roots is 0 or ∞ – the essential-spectrum obstruction of a neutral system (A.6) – and $\operatorname{sech}(\gamma L)$ does not decay on the imaginary axis. Real materials, however, dissipate through *rate-dependent* (Kelvin–Voigt) internal damping, $\gamma = \lambda / \sqrt{1 + c\lambda}$, for which $\gamma \sim \sqrt{\lambda/c}$: the high modes are damped ever harder, $\operatorname{Re} \lambda_k \rightarrow -\infty$, only finitely many roots sit near the axis, and $|\operatorname{sech}(\gamma L)|$ decays like $e^{-\sqrt{\omega/2c}}$. Note how little is needed: the chart uses $c = 0.02$, a light damping that leaves the low modes essentially unaffected and the chart shape intact, yet it is what makes the count exist at all. The chart over (K_p, τ) in Fig. A.10 shows the resulting stable bands (the physically wrong viscous model gives a nonsensical count for the same gains – a diagnostic that the model, not the method, is at fault). The example thus doubles as a reminder that the right damping law is a modelling decision the method faithfully reflects rather than papers over.

A.9 Finite element bar (29 DOF), and the return difference.. The same bar discretized by 29 linear finite elements, with stiffness- proportional (Kelvin–Voigt) damping $\mathbf{C} = c\mathbf{K}$. The naive formulation is the raw determinant $\det(\lambda^2\mathbf{M} + \lambda\mathbf{C} + \mathbf{K} + K_p h^{-1} e^{-\lambda\tau} \mathbf{e}_{29}\mathbf{e}_1^\top)$ (h = element length), and it is a poor one: it grows like λ^{58} , reaches $\sim 10^{39}$ at the top of the integration range, and forces the counting formula onto an estimate of a very large leading order – which leaves an integer residual of 0.12, a rounded count barely better than a coin toss. Factoring out the open-loop dynamics,

$$\mathcal{D}(\lambda) = \det(\mathbf{I} + \mathbf{Q}_0(\lambda)^{-1}\mathbf{F}(\lambda)), \quad \mathbf{Q}_0 = \lambda^2\mathbf{M} + \lambda\mathbf{C} + \mathbf{K}, \quad (\text{A.2})$$

where $\mathbf{F}(\lambda) = K_p h^{-1} e^{-\lambda\tau} \mathbf{e}_{29}\mathbf{e}_1^\top$ is the (rank-one) feedback matrix, leaves the zeros untouched, adds poles only at the open-loop roots – all in the left half-plane, so the right-half-plane count is unaffected – and yields $\mathcal{D} \rightarrow 1$ at infinity, hence $n = 0$ exactly and a residual of $5.0 \cdot 10^{-1}$. Because the feedback here is a single collocated sensor–actuator pair, $\mathbf{F}$ is rank one, and the matrix determinant lemma collapses (A.2) to the classical scalar return difference $1 + L(\lambda)$ of (A.1),

$$\mathcal{D}(\lambda) = 1 + K_p h^{-1} e^{-\lambda\tau} \mathbf{e}_1^\top \mathbf{Q}_0(\lambda)^{-1} \mathbf{e}_{29}, \quad (\text{A.3})$$

which replaces a full 29×29 solve and a 29×29 determinant by one solve with a single right-hand side: it agrees with the determinant form to $2.1 \cdot 10^0$ relative and is 1.8 times cheaper per evaluation. For multi-input feedback of rank r the same lemma leaves an $r \times r$ determinant. This is the large-system economy that Section 8 points to, and it is worth more than any tuning of the integrator. One caveat is inherited from classical loop analysis: an exact pole–zero cancellation between the open loop and the feedback (an unobservable or uncontrollable mode) would remove that

root from (A.2) – as it does from every input–output method – so the split should only be applied when the open-loop model is stabilizable and detectable through the chosen loop. The finite element chart reproduces the continuum result of A.8 within the resolved frequency range: the two are consistency checks on each other. As Section 3.1 argues, where such an open-loop/feedback split exists it is worth more than any tuning of the integrator.

A.10 Large dense determinant.. A synthetic stress test: $\mathcal{D}(\lambda) = \det(\lambda^2 \mathbf{M} + \lambda \mathbf{C} + \mathbf{K} + g e^{-\lambda \tau} \mathbf{K})$ with dense random symmetric positive definite 50×50 matrices, $\mathbf{C} = 0.25 \mathbf{K}$, swept over a delayed stiffness gain $g > -1$ and the delay τ . (At $g = -1$ exactly, the delayed stiffness cancels the static one and a characteristic root sits *on* the integration line; the package then reports the invalid-count marker instead of an arbitrary integer.) The evaluation cost is dominated by the LU factorization per frequency; the adaptive sweep keeps the number of evaluations low, so even this case yields charts in tens of seconds. This example also exercises the overflow guards of Sections 3.1–3.2: $|\mathcal{D}|$ reaches 10^{250} and beyond, so the phase integrand is evaluated in scale-invariant form and the leading-order probe (7) automatically retreats to a representable s_* .

A.11 Fractional-order oscillator.. $\mathcal{D}(\lambda) = \lambda^{1.8} + 0.5 \lambda^{0.8} + k e^{-\lambda \tau}$ over (k, τ) , with the principal branch of the fractional powers. The half-line count applies with the non-integer effective order estimated by (7); the origin offset of Section 3.1 regularizes the branch point at $\lambda = 0$. For $\sigma = 0$ the integration line stays right of the branch cut and the count is unambiguous; shifted lines with $\sigma < 0$ cross the cut of the principal branch, so γ -stability contours of fractional systems refer to the roots of the principal-branch extension of $\mathcal{D}$ (see Section 2.2) – consistent with the convention of the fractional-systems literature [32, 33], but worth keeping in mind when comparing against time-domain simulations.

A.11 Fractional controller chart from the literature.. As an external validation of the fractional capability, Fig. A.11 reproduces the stabilizing-region chart of Gao, Zhai and Liu [77] (Example 1), computed there by D-decomposition; see also the related design charts [78, 79, 80, 81]. The closed-loop characteristic function $\mathcal{D}(s) = s^\mu(10s^{0.5} + 1) + 5e^{-0.4s}(k_p s^\mu + k_i)$ is a one-line model definition for the present method, and the traced boundary and the classification of the published test points agree with the reference.

Why only $\sigma = 0$, and how the violation announces itself.. Section 2.1 sets out why the imaginary axis is admissible for this class while a shifted line is not: the cut of s^μ runs along the negative reals, outside the right half-plane, whereas $\text{Re } \lambda > -\sigma_{\text{deg}}$ with $\sigma_{\text{deg}} > 0$ swallows the segment $[-\sigma_{\text{deg}}, 0]$ of it and the argument principle no longer applies. What is worth recording here is how loudly the figure complains, because it is a useful thing to be able to recognize.

Counts taken on a shifted line must obey a monotonicity that needs no theory to state: moving the line to the left can only ever *add* roots to its right, so $Z(-\sigma)$ is non-decreasing in σ and the γ -stable regions are *nested*. On this system neither holds. A 15×15 scan of both charts returns $Z(-0.5) < Z(0)$ at 3 of 450 points, and an earlier draft of this figure, drawn with several σ_{deg} levels, showed contours that *crossed* – which no nested family can do. The count failed silently; the geometry did not. This is why the stability chart reproduces the literature while the stability-degree chart cannot be offered at all. Extending the criterion to fractional systems with a prescribed decay

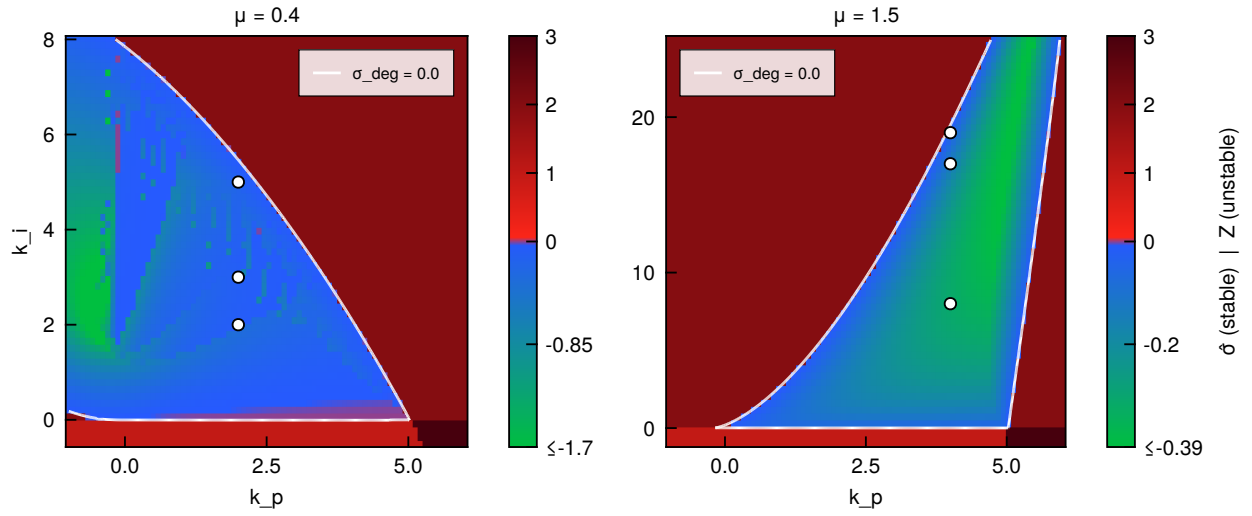


Figure A.11: Reproduction of a fractional-order controller design chart from the literature (Gao, Zhai and Liu [77], Example 1): stabilizing regions of the fractional PI controller $C(s) = k_p + k_i/s^\mu$ for the fractional delayed plant $G(s) = 5e^{-0.4s}/(10s^{0.5} + 1)$, for $\mu = 0.4$ (left) and $\mu = 1.5$ (right). White curve: the traced stability boundary ($\sigma = 0$); white markers: the test points tabulated in [77]. Only $\sigma = 0$ is shown, and deliberately so: the shifted line of Section 2.2 is *not* admissible for this class – see the text.

910 rate requires a contour that respects the cut – a keyhole, or a formulation in the $w = s^{1/m}$ plane – and is left for future work (Section 8).

References

- [1] S. A. Tobias, W. Fishwick, Theory of regenerative machine tool chatter, *The Engineer* 205 (1958) 199–203, 238–239, london.
- 915 [2] J. Tlustý, M. Poláček, The stability of machine tools against self-excited vibrations in machining, in: *International Research in Production Engineering*, ASME, 1963, pp. 465–474.
- [3] Y. Altintas, E. Budak, Analytical prediction of stability lobes in milling, *CIRP Annals - Manufacturing Technology* 44 (1) (1995) 357–362. doi:10.1016/S0007-8506(07)62342-7.
- [4] J. Munoa, X. Beudaert, Z. Dombovari, Y. Altintas, E. Budak, C. Brecher, G. Stépán, Chatter suppression techniques in metal cutting, *CIRP Annals - Manufacturing Technology* 65 (2) (2016) 785–808. doi:10.1016/j.cirp.2016.06.004.
- 920 [5] R. Bellman, K. L. Cooke, *Differential-Difference Equations*, Vol. 6 of *Mathematics in Science and Engineering*, Academic Press, New York, 1963.

- [6] G. Stépán, Retarded Dynamical Systems: Stability and Characteristic Functions, Vol. 210 of Pitman Research Notes in Mathematics Series, Longman Scientific & Technical, Harlow, 1989, co-published with John Wiley & Sons, New York.
- [7] J. K. Hale, S. M. Verduyn Lunel, Introduction to Functional Differential Equations, Vol. 99 of Applied Mathematical Sciences, Springer, New York, 1993. doi:10.1007/978-1-4612-4342-7.
- [8] W. Michiels, S.-I. Niculescu, Stability, Control, and Computation for Time-Delay Systems: An Eigenvalue-Based Approach, 2nd Edition, Advances in Design and Control, SIAM, Philadelphia, PA, 2014. doi:10.1137/1.9781611973631.
- [9] T. Insperger, G. Stépán, Semi-Discretization for Time-Delay Systems: Stability and Engineering Applications, Vol. 178 of Applied Mathematical Sciences, Springer, New York, 2011. doi:10.1007/978-1-4614-0335-7.
- [10] R. Sipahi, S.-I. Niculescu, C. T. Abdallah, W. Michiels, K. Gu, Stability and stabilization of systems with time delay: Limitations and opportunities, IEEE Control Systems Magazine 31 (1) (2011) 38–65. doi:10.1109/MCS.2010.939135.
- [11] T. Insperger, G. Stépán, Semi-discretization method for delayed systems, International Journal for Numerical Methods in Engineering 55 (5) (2002) 503–518. doi:10.1002/nme.505.
- [12] T. Insperger, G. Stépán, Updated semi-discretization method for periodic delay-differential equations with discrete delay, International Journal for Numerical Methods in Engineering 61 (1) (2004) 117–141. doi:10.1002/nme.1061.
- [13] T. Insperger, G. Stépán, J. Turi, On the higher-order semi-discretizations for periodic delayed systems, Journal of Sound and Vibration 313 (1–2) (2008) 334–341. doi:10.1016/j.jsv.2007.11.040.
- [14] Y. Ding, L. Zhu, X. Zhang, H. Ding, A full-discretization method for prediction of milling stability, International Journal of Machine Tools and Manufacture 50 (5) (2010) 502–509. doi:10.1016/j.ijmachtools.2010.01.003.
- [15] E. A. Butcher, H. Ma, E. Bueler, V. Averina, Z. Szabó, Stability of linear time-periodic delay-differential equations via Chebyshev polynomials, International Journal for Numerical Methods in Engineering 59 (7) (2004) 895–922. doi:10.1002/nme.894.
- [16] D. Breda, S. Maset, R. Vermiglio, Pseudospectral differencing methods for characteristic roots of delay differential equations, SIAM Journal on Scientific Computing 27 (2) (2005) 482–495. doi:10.1137/030601600.
- [17] D. Breda, S. Maset, R. Vermiglio, Stability of Linear Delay Differential Equations: A Numerical Approach with MATLAB, SpringerBriefs in Control, Automation and Robotics, Springer, New York, NY, 2015. doi:10.1007/978-1-4939-2107-2.

- [18] F. A. Khasawneh, B. P. Mann, A spectral element approach for the stability of delay systems, *International Journal for Numerical Methods in Engineering* 87 (6) (2011) 566–592. doi:10.1002/nme.3122.
- [19] Z. Wu, W. Michiels, Reliably computing all characteristic roots of delay differential equations in a given right half plane using a spectral method, *Journal of Computational and Applied Mathematics* 236 (9) (2012) 2499–2514. doi:10.1016/j.cam.2011.12.009.
- [20] K. Engelborghs, T. Luzyanina, D. Roose, Numerical bifurcation analysis of delay differential equations using DDE-BIFTOOL, *ACM Transactions on Mathematical Software* 28 (1) (2002) 1–21. doi:10.1145/513001.513002.
- [21] J. Sieber, K. Engelborghs, T. Luzyanina, G. Samaey, D. Roose, DDE-BIFTOOL manual — bifurcation analysis of delay differential equations, arXiv:1406.7144v4 (2016). arXiv:1406.7144.
URL <https://arxiv.org/abs/1406.7144>
- [22] D. Breda, S. Maset, R. Vermiglio, TRACE-DDE: a tool for robust analysis and characteristic equations for delay differential equations, in: J. J. Loiseau, W. Michiels, S.-I. Niculescu, R. Sipahi (Eds.), *Topics in Time Delay Systems: Analysis, Algorithms and Control*, Vol. 388 of *Lecture Notes in Control and Information Sciences*, Springer, Berlin, Heidelberg, 2009, pp. 145–155. doi:10.1007/978-3-642-02897-7_13.
- [23] R. Szalai, G. Stépán, S. J. Hogan, Continuation of bifurcations in periodic delay-differential equations using characteristic matrices, *SIAM Journal on Scientific Computing* 28 (4) (2006) 1301–1317. doi:10.1137/040618709.
- [24] R. Szalai, Knut: a continuation and bifurcation software for delay-differential equations, source code at <https://github.com/rs1909/knut>, accessed 2026-07-16 (2013).
URL <https://rs1909.github.io/knut/>
- [25] D. Bachrathy, H. T. Sykora, SemiDiscretizationMethod.jl: A Julia package to investigate the behaviour of linear delay differential equations, version 0.4.1, accessed 2026-07-16 (2026).
URL <https://github.com/bachrathyd/SemiDiscretizationMethod.jl>
- [26] D. Bachrathy, Linear time complexity analysis of time-periodic delayed systems with multiplication free semi-discretization method, *Journal of Vibration and Control* Published online 22 May 2026 (2026). doi:10.1177/10775463261453846.
- [27] J. K. Hale, S. M. Verduyn Lunel, Strong stabilization of neutral functional differential equations, *IMA Journal of Mathematical Control and Information* 19 (1–2) (2002) 5–23. doi:10.1093/imamci/19.1_and_2.5.
- [28] W. Michiels, T. Vyhldal, An eigenvalue based approach for the stabilization of linear time-delay systems of neutral type, *Automatica* 41 (6) (2005) 991–998. doi:10.1016/j.automatica.2004.11.032.

- [29] S. L. Campbell, Singular Systems of Differential Equations, Vol. 40 of Research Notes in Mathematics, Pitman, San Francisco, 1980.
- [30] P. Ha, V. Mehrmann, Analysis and numerical solution of linear delay differential-algebraic equations, *BIT Numerical Mathematics* 56 (2) (2016) 633–657. doi:10.1007/s10543-015-0577-6.
- 990 [31] J. M. Cushing, Integrodifferential Equations and Delay Models in Population Dynamics, Vol. 20 of Lecture Notes in Biomathematics, Springer, Berlin, Heidelberg, 1977. doi:10.1007/978-3-642-93073-7.
- [32] I. Petráš, Fractional-Order Nonlinear Systems: Modeling, Analysis and Simulation, Nonlinear Physical Science, Springer, Berlin, Heidelberg, 2011. doi:10.1007/978-3-642-18101-6.
- [33] A. R. Fioravanti, C. Bonnet, H. Özbay, S.-I. Niculescu, A numerical method for stability windows and unstable root-locus calculation for linear fractional time-delay systems, *Automatica* 48 (11) (2012) 2824–2830. doi:10.1016/j.automatica.2012.04.009.
- 995 [34] J. Čermák, T. Kisela, Delay-dependent stability switches in fractional differential equations, *Communications in Nonlinear Science and Numerical Simulation* 79 (2019) 104888. doi:10.1016/j.cnsns.2019.104888.
- [35] K. Gu, V. L. Kharitonov, J. Chen, Stability of Time-Delay Systems, Birkhäuser, Boston, 2003. doi:10.1007/978-1-4612-0039-0.
- 1000 [36] J. I. Neimark, D-decomposition of the space of quasi-polynomials (on the stability of linearized distributive systems), *Prikladnaya Matematika i Mekhanika* 13 (1949) 349–380, in Russian. English translation: American Mathematical Society Translations, Series 2, vol. 102, pp. 95–131, 1973.
- [37] H. Nyquist, Regeneration theory, *Bell System Technical Journal* 11 (1) (1932) 126–147. doi:10.1002/j.1538-7305.1932.tb02344.x.
- 1005 [38] A. V. Mikhailov, Metod garmonicheskogo analiza v teorii regulirovaniya (Method of harmonic analysis in control theory), *Avtomatika i Telemekhanika* (3) (1938) 27–81, in Russian.
- [39] K. Walton, J. E. Marshall, Direct method for TDS stability analysis, *IEE Proceedings D – Control Theory and Applications* 134 (2) (1987) 101–107. doi:10.1049/ip-d.1987.0018.
- 1010 [40] Z. V. Rekasius, A stability test for systems with delays, in: Proceedings of the Joint Automatic Control Conference, San Francisco, CA, 1980, paper No. TP9-A.
- [41] N. Olgac, R. Sipahi, An exact method for the stability analysis of time-delayed linear time-invariant (LTI) systems, *IEEE Transactions on Automatic Control* 47 (5) (2002) 793–797. doi:10.1109/TAC.2002.1000275.

- [42] X.-G. Li, S.-I. Niculescu, A. Çela, Analytic Curve Frequency-Sweeping Stability Tests for Systems with Commensurate Delays, SpringerBriefs in Electrical and Computer Engineering – Control, Automation and Robotics, Springer, Cham, 2015. doi : 10.1007/978-3-319-15717-7.
- [43] S. Yi, P. W. Nelson, A. G. Ulsoy, Time-Delay Systems: Analysis and Control Using the Lambert W Function, World Scientific, Singapore, 2010. doi : 10.1142/7759.
- [44] J. Louisell, A matrix method for determining the imaginary axis eigenvalues of a delay system, IEEE Transactions on Automatic Control 46 (12) (2001) 2008–2012. doi : 10.1109/9.975510.
- [45] T. Vyhlídal, P. Zítek, Mapping based algorithm for large-scale computation of quasi-polynomial zeros, IEEE Transactions on Automatic Control 54 (1) (2009) 171–177. doi : 10.1109/TAC.2008.2008345.
- [46] T. Vyhlídal, P. Zítek, QPmR – Quasi-Polynomial Root-Finder: Algorithm update and examples, in: T. Vyhlídal, J.-F. Lafay, R. Sipahi (Eds.), Delay Systems: From Theory to Numerics and Applications, Vol. 1 of Advances in Delays and Dynamics, Springer, Cham, 2014, pp. 299–312. doi : 10.1007/978-3-319-01695-5_22.
- [47] W.-J. Beyn, An integral method for solving nonlinear eigenvalue problems, Linear Algebra and its Applications 436 (10) (2012) 3839–3863. doi : 10.1016/j.laa.2011.03.030.
- [48] D. Avanesoff, A. R. Fioravanti, C. Bonnet, YALTA: a Matlab toolbox for the H_∞ -stability analysis of classical and fractional systems with commensurate delays, in: 11th IFAC Workshop on Time-Delay Systems, Vol. 46 of IFAC Proceedings Volumes, 2013, pp. 839–844. doi : 10.3182/20130204-3-FR-4032.00088.
- [49] S.-I. Niculescu, I. Boussaada, Counting characteristic roots of linear delay differential equations. Part II: From argument principle to rightmost root assignment methods, in: D. Breda (Ed.), Controlling Delayed Dynamics: Advances in Theory, Methods and Applications, Vol. 604 of CISM International Centre for Mechanical Sciences, Springer, Cham, 2023, pp. 157–193. doi : 10.1007/978-3-031-01129-0_6.
- [50] D. Bachrathy, G. Stépán, Bisection method in higher dimensions and the efficiency number, Periodica Polytechnica Mechanical Engineering 56 (2) (2012) 81–86. doi : 10.3311/pp.me.2012-2.01.
- [51] D. Bachrathy, MDBM.jl: Multi-dimensional bisection method, version 0.1.7 (2025).
URL <https://github.com/bachrathyd/MDBM.jl>
- [52] G. Stépán, Modelling nonlinear regenerative effects in metal cutting, Philosophical Transactions of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences 359 (1781) (2001) 739–757. doi : 10.1098/rsta.2000.0753.
- [53] T. Insperger, B. P. Mann, G. Stépán, P. V. Bayly, Stability of up-milling and down-milling, part 1: Alternative analytical methods, International Journal of Machine Tools and Manufacture 43 (1) (2003) 25–34. doi : 10.1016/S0890-6955(02)00159-1.

- 1045 [54] D. Bachrathy, G. Stépán, Improved prediction of stability lobes with extended multi frequency solution, *CIRP Annals - Manufacturing Technology* 62 (1) (2013) 411–414. doi:10.1016/j.cirp.2013.03.085.
- [55] V. B. Kolmanovskii, A. Myshkis, *Introduction to the Theory and Applications of Functional Differential Equations*, Vol. 463 of *Mathematics and Its Applications*, Kluwer Academic Publishers, Dordrecht, 1999. doi:10.1007/978-94-017-1965-0.
- 1050 [56] L. M. Delves, J. N. Lyness, A numerical method for locating the zeros of an analytic function, *Mathematics of Computation* 21 (100) (1967) 543–560. doi:10.1090/S0025-5718-1967-0228165-4.
- [57] P. Kravanja, M. Van Barel, *Computing the Zeros of Analytic Functions*, Vol. 1727 of *Lecture Notes in Mathematics*, Springer, Berlin, Heidelberg, 2000. doi:10.1007/BFb0103927.
- [58] P. Kravanja, M. Van Barel, O. Ragos, M. N. Vrahatis, F. A. Zafiropoulos, ZEAL: A mathematical software
1055 package for computing zeros of analytic functions, *Computer Physics Communications* 124 (2–3) (2000) 212–232. doi:10.1016/S0010-4655(99)00429-4.
- [59] B. D. Hassard, Counting roots of the characteristic equation for linear delay-differential systems, *Journal of Differential Equations* 136 (2) (1997) 222–235. doi:10.1006/jdeq.1996.3127.
- [60] P. Zítek, T. Vyhlídal, Argument-increment based stability criterion for neutral time delay systems, in: *Proceedings of the 16th Mediterranean Conference on Control and Automation*, 2008, pp. 824–829. doi:
1060 10.1109/MED.2008.4602199.
- [61] T. Vyhlídal, P. Zítek, Modification of Mikhaylov criterion for neutral time-delay systems, *IEEE Transactions on Automatic Control* 54 (10) (2009) 2430–2435. doi:10.1109/TAC.2009.2029301.
- [62] Q. Xu, Z. Wang, Exact stability test of neutral delay differential equations via a rough estimation of the
1065 testing integral, *International Journal of Dynamics and Control* 2 (2) (2014) 154–163. doi:10.1007/s40435-013-0044-7.
- [63] Q. Xu, G. Stépán, Z. Wang, Delay-dependent stability analysis by using delay-independent integral evaluation, *Automatica* 70 (2016) 153–157. doi:10.1016/j.automatica.2016.03.028.
- [64] J. H. Verner, Numerically optimal Runge–Kutta pairs with interpolants, *Numerical Algorithms* 53 (2–3) (2010)
1070 383–396. doi:10.1007/s11075-009-9290-3.
- [65] C. Tsitouras, Runge–Kutta pairs of order 5(4) satisfying only the first column simplifying assumption, *Computers & Mathematics with Applications* 62 (2) (2011) 770–775. doi:10.1016/j.camwa.2011.06.002.
- [66] C. Rackauckas, Q. Nie, DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia, *Journal of Open Research Software* 5 (1) (2017) 15. doi:10.5334/jors.151.

- 1075 [67] J. Revels, M. Lubin, T. Papamarkou, Forward-mode automatic differentiation in Julia (2016). arXiv:1607.07892.
URL <https://arxiv.org/abs/1607.07892>
- [68] J. Bezanson, A. Edelman, S. Karpinski, V. B. Shah, Julia: A fresh approach to numerical computing, *SIAM Review* 59 (1) (2017) 65–98. doi:10.1137/141000671.
- 1080 [69] L. Zhang, G. Stépán, Exact stability chart of an elastic beam subjected to delayed feedback, *Journal of Sound and Vibration* 367 (2016) 219–232. doi:10.1016/j.jsv.2016.01.002.
- [70] W. Michiels, Spectrum-based stability analysis and stabilisation of systems described by delay differential algebraic equations, *IET Control Theory & Applications* 5 (16) (2011) 1829–1842. doi:10.1049/iet-cta.2010.0752.
- 1085 [71] S. G. Johnson, QuadGK.jl: Gauss–Kronrod integration in Julia, version 2.11.3 (2013).
URL <https://github.com/JuliaMath/QuadGK.jl>
- [72] H. H. Rosenbrock, Some general implicit processes for the numerical solution of differential equations, *The Computer Journal* 5 (4) (1963) 329–330. doi:10.1093/comjnl/5.4.329.
- [73] L. F. Shampine, M. W. Reichelt, The MATLAB ODE suite, *SIAM Journal on Scientific Computing* 18 (1) (1997) 1–22. doi:10.1137/S1064827594276424.
- 1090 [74] E. Hairer, G. Wanner, Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems, 2nd Edition, Vol. 14 of Springer Series in Computational Mathematics, Springer, Berlin, Heidelberg, 1996. doi:10.1007/978-3-642-05221-7.
- [75] D. J. VandenHeuvel, DelaunayTriangulation.jl: A Julia package for Delaunay triangulations and Voronoi tessellations in the plane, *Journal of Open Source Software* 9 (101) (2024) 7174. doi:10.21105/joss.07174.
- 1095 [76] M. Kidd, G. Stépán, Delayed control of an elastic beam, *International Journal of Dynamics and Control* 2 (1) (2014) 68–76. doi:10.1007/s40435-014-0079-4.
- [77] Z. Gao, L.-R. Zhai, Y.-D. Liu, Robust stabilizing regions of fractional-order PI^l controllers for fractional-order systems with time-delays, *International Journal of Automation and Computing* 14 (3) (2017) 340–349. doi:10.1007/s11633-015-0941-7.
- 1100 [78] S. E. Hamamci, An algorithm for stabilization of fractional-order time delay systems using fractional-order PID controllers, *IEEE Transactions on Automatic Control* 52 (10) (2007) 1964–1969. doi:10.1109/TAC.2007.906243.

- 1105 [79] N. Hohenbichler, Comments on “An algorithm for stabilization of fractional-order time delay systems using fractional-order PID controllers”, IEEE Transactions on Automatic Control 53 (11) (2008) 2712. doi:10.1109/TAC.2008.2007535.
- [80] A. Ruszewski, Stability regions of closed loop system with time delay inertial plant of fractional order and fractional order PI controller, Bulletin of the Polish Academy of Sciences: Technical Sciences 56 (4) (2008) 329–332.
- 1110 [81] Y.-C. Cheng, C. Hwang, Stabilization of unstable first-order time-delay systems using fractional-order PD controllers, Journal of the Chinese Institute of Engineers 29 (2) (2006) 241–249. doi:10.1080/02533839.2006.9671121.

Table 9: Comparison with semi-discretization (`SemiDiscretizationMethod.jl`, orders 1 and 2, n steps per delay) on the reduced showcase system: error of the dominant characteristic exponent and CPU time per parameter point. The reference is the Newton-polished dominant root of the quasi-polynomial itself, certified independently of either sweep by its recorded defect $|\mathcal{D}(\lambda_{\text{ref}})| < 10^{-12}$. The unrefined rows report the accuracy of the tracked estimate alone; the refined rows share the polishing machinery with the reference, so their error reflects the polish rather than the sweep.

point	method / setting	error of $\text{Re } \lambda_{\text{dom}}$	time / point
stable	SD order 1, $n = 20$	$2.9 \cdot 10^{-5}$	4.5 ms
	SD order 1, $n = 160$	$4.6 \cdot 10^{-7}$	435.8 ms
	SD order 2, $n = 20$	$2.0 \cdot 10^{-6}$	5.4 ms
	SD order 2, $n = 160$	$4.0 \cdot 10^{-9}$	472.7 ms
	this work, tol 10^{-5} , unrefined	$2.7 \cdot 10^{-4}$	1.4 ms
	this work, tol 10^{-5}	$6.2 \cdot 10^{-17}$	1.4 ms
	this work, tol 10^{-8}	$3.1 \cdot 10^{-17}$	3.1 ms
near boundary	SD order 1, $n = 20$	$9.3 \cdot 10^{-5}$	4.5 ms
	SD order 1, $n = 160$	$1.5 \cdot 10^{-6}$	417.2 ms
	SD order 2, $n = 20$	$4.0 \cdot 10^{-6}$	4.8 ms
	SD order 2, $n = 160$	$7.9 \cdot 10^{-9}$	477.4 ms
	this work, tol 10^{-5} , unrefined	$2.7 \cdot 10^{-4}$	1.7 ms
	this work, tol 10^{-5}	$6.9 \cdot 10^{-18}$	1.8 ms
	this work, tol 10^{-8}	$6.9 \cdot 10^{-18}$	3.6 ms
unstable	SD order 1, $n = 20$	$9.7 \cdot 10^{-5}$	9.9 ms
	SD order 1, $n = 160$	$1.5 \cdot 10^{-6}$	424.3 ms
	SD order 2, $n = 20$	$2.5 \cdot 10^{-7}$	9.3 ms
	SD order 2, $n = 160$	$5.6 \cdot 10^{-10}$	450.7 ms
	this work, tol 10^{-5} , unrefined	$3.6 \cdot 10^{-2}$	1.2 ms
	this work, tol 10^{-5}	$5.5 \cdot 10^{-11}$	1.2 ms
	this work, tol 10^{-8}	$4.1 \cdot 10^{-12}$	2.8 ms

Table 10: Evaluations of $\mathcal{D}$ needed for one parameter point at $\text{tol} = 10^{-5}$, with the count of each entry verified against a tight-tolerance reference before being recorded. What governs the cost is how fast the delay-induced ripple of θ' decays, i.e. how far the delayed terms sit below the leading power: ω^{-1} for the velocity-feedback showcase, ω^{-3} for the fourth-order position-feedback benchmark.

system	back-end	$\omega_{\max} = 10^4$	$\omega_{\max} = 10^6$
showcase (ω^{-1} ripple)	ordered march	11 138	578 418
	quadrature	15 315	1 675 500
4th-order (ω^{-3} ripple)	ordered march	1 330	1 426
	quadrature	555	570